

Universidad Galileo

Definición del Alcance del Proyecto de Software a desarrollar

GESTIÓN DE PAGOS MENSUALES TRANSPORTES GÉNESIS

Tipo de proyecto:

Mantenimiento

Versión: 1.0

Nombre Autorizador	Firma	Fecha
Jonathan Samuel Villeda Pérez	JONATHAN VILLEDA	12/05/2026
José David Florian	DAVID FLORIÁN	12/05/2026

Contenido

ESPECIFICACIÓN FUNCIONAL - TRANSPORTES GENESIS	5
Sistema de Gestión de Transporte Escolar.....	5
1.1 Introducción y alcance del sistema.....	5
1.1.1 Descripción General	5
1.1.2 Problema que Resuelve.....	5
1.1.3 Objetivos del Proyecto.....	6
1.1.4 Alcance del Sistema	6
1.1.5 Beneficios Esperados	8
1.1.6 Usuarios del Sistema	9
1.2 Contexto organizacional y stakeholders.....	9
1.2.1 Modelo Organizacional de Empresa de Transporte Escolar.....	9
1.2.2 Stakeholders del Proyecto	10
1.2.3 Flujo Operativo Actual vs Propuesto	11
1.3 Análisis de Requerimientos Funcionales.....	17
1.3.1 Requerimientos Funcionales por Módulo.....	17
1.3.2 Matriz de Trazabilidad (Requerimientos ↔ Casos de Uso)	25
• Autenticación (CU-01, CU-02)	27
• Gestión de Alumnos (CU-05, CU-06)	27
• Gestión de Paradas (CU-07, CU-08).....	28
• Rutas (CU-09, CU-10).....	28
• Geolocalización (CU-11, CU-12).....	28
• Pagos (CU-13, CU-14, CU-15).....	29
• Asistencia (CU-16)	29
• Recogidas (CU-17).....	30
• Dashboard y Reportes (CU-18, CU-19)	30
1.4 Análisis de Requerimientos No Funcionales.....	31
1.4.1 Requerimientos de Rendimiento (Performance)	31
1.4.2 Requerimientos de Usabilidad.....	32
1.4.3 Requerimientos de Seguridad	33
1.4.4 Requerimientos de Disponibilidad (Availability).....	34
1.4.5 Requerimientos de Escalabilidad.....	34
1.4.6 Requerimientos de Mantenibilidad.....	35

1.4.7 Requerimientos de Compatibilidad	35
1.4.8 Requerimientos de Cumplimiento Legal.....	36
1.5 Análisis de Dominio	37
1.5.1 Modelo Conceptual del Negocio.....	37
1.5.2 Reglas de Negocio.....	40
1.5.3 Procesos de Negocio Clave.....	42
1.6 Supuestos, restricciones y dependencias funcionales.....	45
1.6.1 Supuestos del Proyecto	45
1.6.2 Restricciones del Proyecto.....	46
1.6.3 Dependencias Funcionales	48
1.6.4 Riesgos Funcionales	49
1.1.5 Usuarios Finales del Sistema	50
1.1.6 Beneficios Esperados	51
ESPECIFICACIÓN TÉCNICA - TRANSPORTES GENESIS	51
2.1 Arquitectura de referencia.....	51
2.1.1 Diagrama C4 - Nivel 1: Contexto del Sistema	51
2.1.2 Diagrama C4 - Nivel 2: Contenedores.....	52
2.1.3 Diagrama C4 - Nivel 3: Componentes (Web Application).....	54
2.1.4 Estilo Arquitectónico y Justificación	56
2.1.5 Diagrama de Despliegue	58
2.2 Stack tecnológico seleccionado	60
2.2.1 Tecnologías por Capa.....	60
2.2.4 Trade-offs y Decisiones Técnicas	63
2.3 Requerimientos de infraestructura y entornos	64
2.3.1 Hardware Mínimo y Recomendado (On-Premise).....	64
2.3.3 Diagrama de Red y Topología de Seguridad	65
2.4 Diseño de base de datos	66
2.4.2 Diagrama Entidad-Relación (ER)	66
2.5 Diseño de APIs e integración.....	69
2.5.1 Definiciones de los APIs y Endpoints REST Principales implementados:	69
2.6 Patrones de diseño y buenas prácticas	71
2.6.1 Estrategia de Testing.....	71
2.7 Plan de implementación y roadmap técnico	72
2.7.1 Fases del Proyecto (Completadas)	72

2.8 Análisis de riesgos técnicos y plan de mitigación	73
2.8.1 Matriz de Riesgos Técnicos	73
3. ESPECIFICACIÓN ECONÓMICA	75
3.1 Resumen Ejecutivo Económico	75
3.2 Análisis de Costos (Cost Breakdown Structure)	75
3.2.1 Costos de Desarrollo	75
3.2.2 Costos de Infraestructura.....	75
3.2.3 Costos de Capacitación	76
3.2.4 Costos de Mantenimiento.....	76
3.2.5 Total General del Proyecto	76
3.3 Estimación de Esfuerzo y Duración	76
Rango de estimación:.....	76
3.4 Modelo de Negocio	77
Tipo de modelo:	77
Propuesta de valor:.....	77
Estrategia futura:.....	77
3.5 Análisis Financiero.....	77
Flujo de retorno (estimado)	77
Indicadores:.....	77
3.6 Análisis de Mercado y Competencia	77
Contexto:.....	77
Oportunidad:.....	77
3.7 Plan de Negocio Ejecutivo	78
3.8 Fuentes de Financiamiento	78
3.9 Análisis de Riesgos Económicos.....	78
Beneficios del Proyecto.....	78
Beneficios Tangibles	78
Beneficios Intangibles	78

ESPECIFICACIÓN FUNCIONAL - TRANSPORTES GENESIS

Sistema de Gestión de Transporte Escolar

Proyecto: Transportes Genesis

Tipo de Documento: Especificación Funcional

Versión: 1.0

Fecha: 2025

Autor: Equipo de Desarrollo TransportesGenesis

Repositorio: <https://github.com/johnsvill/TransportesGenesis>

Rama: dev_intermedia

1.1 Introducción y alcance del sistema

1.1.1 Descripción General

Transportes Genesis es un sistema integral de gestión de transporte escolar desarrollado en .NET 8 con Razor Pages que digitaliza y optimiza las operaciones diarias de empresas de transporte escolar. El sistema proporciona una plataforma centralizada que conecta a todos los actores involucrados en el servicio de transporte: administradores, pilotos, monitores y padres de familia.

1.1.2 Problema que Resuelve

Las empresas de transporte escolar tradicionalmente operan con procesos manuales que generan:

- **Ineficiencia operativa:** Planificación de rutas manual y sin optimización
- **Falta de visibilidad:** Los padres no saben dónde está el bus en tiempo real
- **Desorganización:** Registros en papel de recogidas, asistencias y pagos
- **Baja satisfacción del cliente:** Poca comunicación y transparencia
- **Riesgos de seguridad:** Falta de trazabilidad de quién recoge a cada alumno
- **Costos elevados:** Rutas no optimizadas consumen más combustible
- **Errores humanos:** Pérdida de registros, pagos no confirmados, asistencias mal registradas

1.1.3 Objetivos del Proyecto

Objetivos Generales

1. **Digitalizar** todas las operaciones de transporte escolar en una plataforma web centralizada
2. **Optimizar** las rutas mediante algoritmos de cálculo automático (TSP - Traveling Salesman Problem)
3. **Proporcionar visibilidad en tiempo real** de la ubicación de los buses mediante geolocalización
4. **Mejorar la comunicación** entre la empresa de transporte y los padres de familia
5. **Aumentar la seguridad** con registro detallado de recogidas y asistencias
6. **Reducir costos operativos** mediante rutas optimizadas y procesos automatizados

Objetivos Específicos

1. Implementar un sistema de autenticación y autorización basado en roles
2. Crear un módulo de cálculo automático de rutas optimizadas
3. Desarrollar un sistema de geolocalización en tiempo real con SignalR
4. Proveer un módulo de gestión de pagos para padres de familia
5. Implementar un sistema de confirmación de asistencia diaria
6. Crear un módulo de registro de recogidas para monitores
7. Proporcionar dashboards y reportes para administradores

1.1.4 Alcance del Sistema

Dentro del Alcance (v1.0)

El sistema SÍ incluye:

1. **Gestión de Usuarios y Roles**
 - Login/Logout con ASP.NET Core Identity
 - Roles: Administrador, Piloto, Monitor, Padre de Familia
 - Gestión de perfiles de usuario
 - Cambio de contraseña (obligatorio en primer login)
2. **Gestión de Buses y Asignaciones**
 - CRUD de buses (placa, modelo, capacidad)
 - Asignación de pilotos a buses
 - Asignación de monitores a buses
 - Historial de asignaciones
3. **Gestión de Alumnos**
 - CRUD de alumnos
 - Vinculación con padres de familia
 - Información básica: nombre, fecha de nacimiento, grado

4. **Gestión de Paradas**
 - CRUD de paradas
 - Geolocalización de paradas (latitud, longitud)
 - Direcciones descriptivas
5. **Cálculo y Gestión de Rutas**
 - Cálculo automático de rutas optimizadas (algoritmo TSP)
 - Rutas diferenciadas por turno: Mañana y Tarde
 - Asignación de paradas a rutas con orden secuencial
 - Estimación de horarios por parada
 - Visualización de rutas para pilotos y monitores
6. **Geolocalización en Tiempo Real**
 - Envío de ubicación del bus cada X segundos (SignalR)
 - Mapa en tiempo real para visualizar buses
 - Histórico de ubicaciones (opcional en v1.0)
7. **Módulo de Pagos**
 - Registro de pagos realizados por padres
 - Historial de pagos
 - Estado de pagos (Pendiente, Confirmado)
 - Métodos de pago (efectivo, transferencia, etc.)
8. **Confirmación de Asistencia**
 - Los padres pueden confirmar si el alumno asistirá o no
 - Confirmación diaria
 - Visualización del estado de confirmación
9. **Registro de Recogidas (Monitor)**
 - El monitor registra qué alumnos fueron recogidos en cada parada
 - Prevención de duplicados
 - Registro de fecha/hora y ubicación de recogida
 - Visualización del estado de recogidas
10. **Dashboards y Reportes Básicos**
 - Panel de administración con estadísticas básicas
 - Reportes de pagos
 - Reportes de asistencia

Fuera del Alcance (v1.0)

El sistema **NO incluye** en la versión inicial:

1. **Aplicación móvil nativa** (iOS/Android)
 - Solo interfaz web responsive (funciona en móviles vía navegador)
2. **Notificaciones push**
 - No se envían notificaciones automáticas a dispositivos móviles
 - Posible en versiones futuras con Firebase/OneSignal

3. **Chat en tiempo real** entre usuarios
 - No hay mensajería interna en el sistema
4. **Gestión de mantenimiento de buses**
 - Registro de mantenimientos preventivos/correctivos no incluido en v1.0
5. **Módulo de quejas y reclamos formal**
 - No hay sistema de tickets de soporte en v1.0
6. **Análisis avanzado de datos (BI)**
 - Dashboards básicos únicamente; no hay herramientas de Business Intelligence
7. **Multi-idioma**
 - El sistema está únicamente en español

1.1.5 Beneficios Esperados

Para la Empresa de Transporte

-  Reducción del 20-30% en costos de combustible gracias a rutas optimizadas
-  Ahorro de 10-15 horas/semana en tareas administrativas
-  Mayor control operativo con visibilidad en tiempo real de toda la flota
-  Reducción de errores en registros de pagos y asistencias
-  Mejora en la imagen corporativa al ofrecer tecnología moderna

Para los Pilotos

-  Rutas claras y optimizadas sin necesidad de planificación manual
-  Información actualizada de alumnos que asistirán cada día
-  Interfaz simple para ver su ruta diaria

Para los Monitores

-  Proceso digitalizado de registro de recogidas (adiós al papel)
-  Interfaz intuitiva para marcar asistencias en tiempo real

Para los Padres de Familia

-  Tranquilidad al ver dónde está el bus en tiempo real
-  Comunicación efectiva al confirmar asistencia de su hijo
-  Transparencia en el estado de pagos
-  Accesibilidad desde cualquier dispositivo con navegador

1.1.6 Usuarios del Sistema

Rol	Descripción	Cantidad Estimada	Funcionalidades Principales
Administrador	Dueño o gerente de la empresa de transporte	1-3 por empresa	Gestión completa del sistema, reportes, configuración
Piloto	Conductor del bus	5-50 por empresa	Ver su ruta diaria, actualizar ubicación
Monitor	Acompañante en el bus que supervisa a los alumnos	5-50 por empresa	Ver ruta, registrar recogidas, marcar asistencias
Padre de Familia	Tutor del alumno	50-500 por empresa	Ver ubicación del bus, confirmar asistencia, ver pagos

✓ 1.1 INTRODUCCIÓN Y ALCANCE DEL SISTEMA - COMPLETADO

1.2 Contexto organizacional y stakeholders

1.2.1 Modelo Organizacional de Empresa de Transporte Escolar

Estructura Organizacional Típica - Empresa de Transporte Escolar

Nivel 1 y 2: Dirección General / Dueño /Administrador / Gerente de Operaciones

- Nivel 3: Coordinador de Rutas
- Nivel 3: Encargado de Finanzas/Pagos
- Nivel 3: Supervisor de Flota
- Nivel 3: Pilotos (Conductores)
 - └─ Asignados a buses específicos
- Nivel 3: Monitores
 - └─ Asignados a buses específicos

Relación externa:

- Padres de Familia → Clientes del servicio
- Alumnos → Usuarios finales del transporte

Interacción con el sistema:

- Administrador gestiona todo
- Pilotos y Monitores operan el servicio diario
- Padres consultan información y realizan pagos

1.2.2 Stakeholders del Proyecto

Stakeholders Primarios (Usuarios Directos)

Stakeholder	Rol en el Negocio	Interés en el Sistema	Influencia	Expectativas
Dueño de la empresa / Administrador	Inversionista principal/ Gerente de operaciones	Alto	● Crítica	ROI, eficiencia, reducción de costos
Pilotos	Operadores de buses	Medio	● Media	Interfaz simple, rutas claras
Monitores	Supervisores de alumnos	Medio	● Media	Registro rápido, fácil de usar en movimiento
Padres de Familia	Clientes pagadores	Alto	● Baja	Visibilidad, transparencia, seguridad de sus hijos
Alumnos	Usuarios finales (pasivos)	Bajo	● Baja	No interactúan directamente con el sistema

1.2.3 Flujo Operativo Actual vs Propuesto

Proceso Actual (Manual)

Flujo: Planificación de Rutas

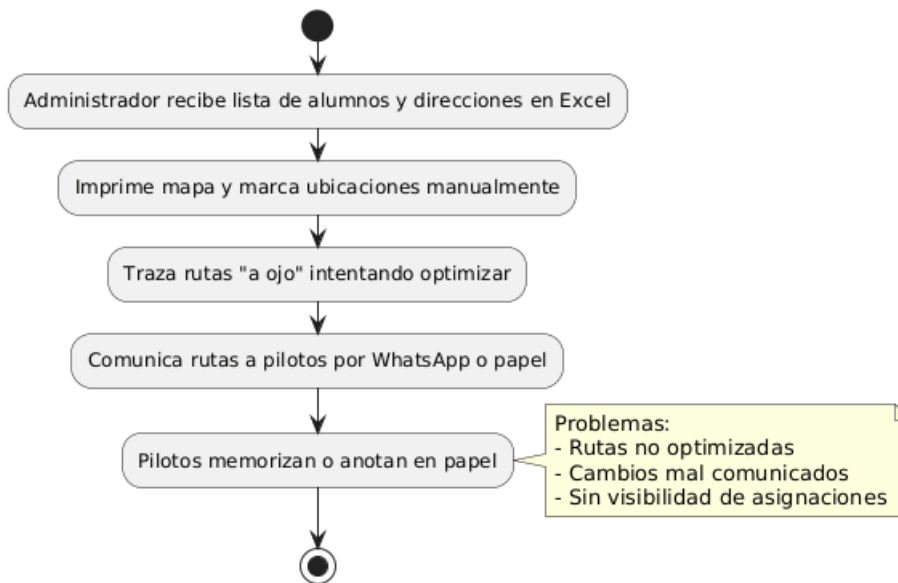
[Administrador]

- ↓
1. Recibe lista de alumnos y direcciones en Excel
- ↓
2. Imprime mapa y marca ubicaciones manualmente
- ↓
3. Traza rutas "a ojo" intentando optimizar
- ↓
4. Comunica rutas a pilotos por WhatsApp o papel
- ↓
5. Pilotos memorizan o anotan en papel

Problemas:

- Rutas no optimizadas (más tiempo y combustible)
- Cambios no se comunican eficientemente
- Sin visibilidad de quién recoge a quién

Proceso Actual (Manual) - Planificación de Rutas



Flujo: Registro de Recogidas

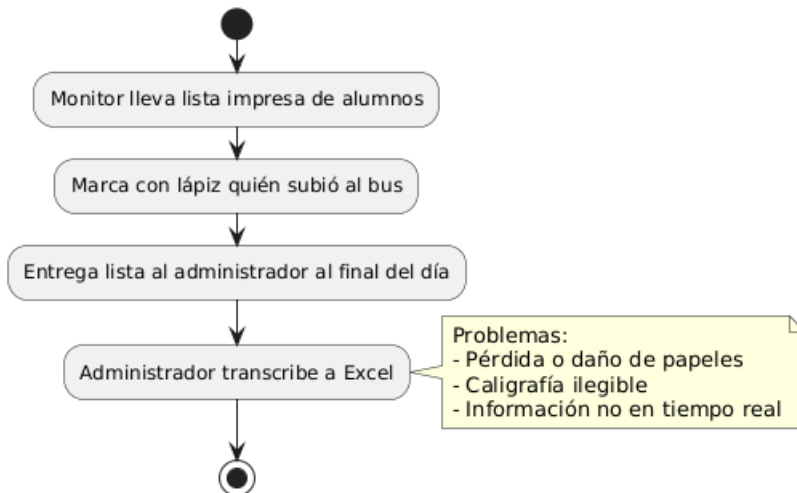
[Monitor]

- ↓
1. Lleva lista impresa de alumnos por parada
- ↓
2. Marca con lápiz quién subió al bus
- ↓
3. Al final del día, entrega lista al administrador
- ↓
4. Administrador transcribe a Excel (doble trabajo)

Problemas:

- Papeles se pierden o mojan
- Caligrafía ilegible
- Retraso en información (no es en tiempo real).

Proceso Actual (Manual) - Registro de Recogidas



Flujo: Control de Pagos

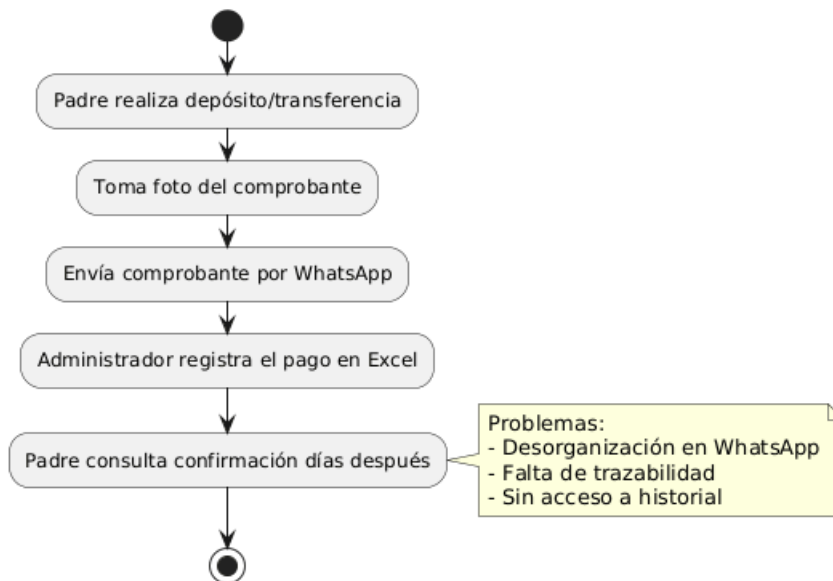
[Padre de Familia]

- ↓
1. Hace depósito/transferencia bancaria
- ↓
2. Toma foto del comprobante, envía por WhatsApp al administrador
- ↓
3. Administrador anota en Excel manualmente
- ↓
4. Padre pregunta si se confirmó el pago (días después)

Problemas:

- Desorganización en WhatsApp (mensajes se pierden)
- No hay trazabilidad clara
- Padre no tiene acceso a su historial de pagos

Proceso Actual (Manual) - Control de Pagos



Proceso Propuesto (Digitalizado con Transportes Genesis)

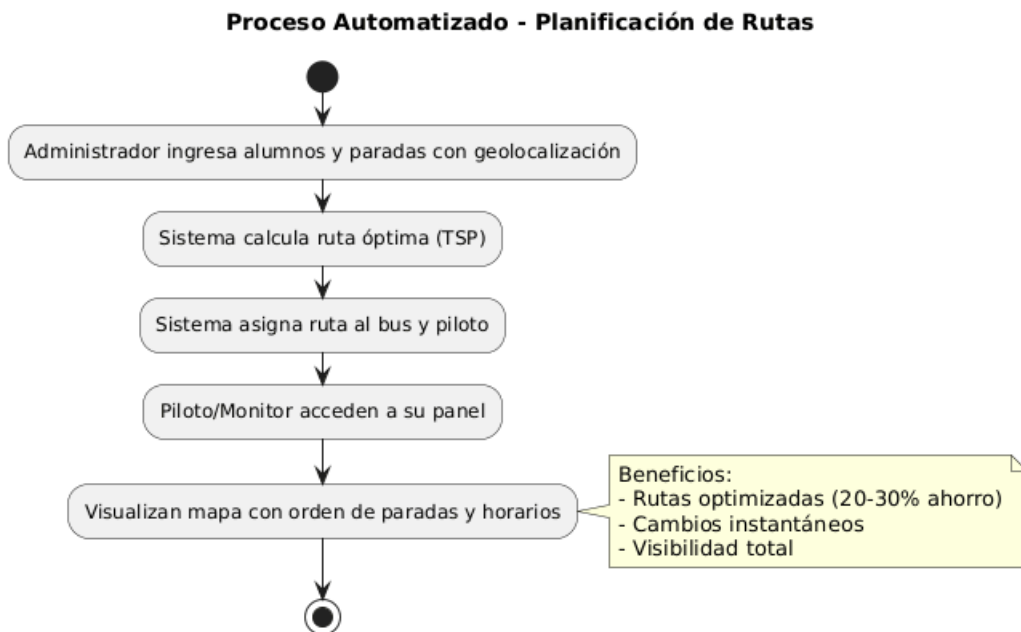
Flujo: Planificación de Rutas

[Administrador en el sistema]

- ↓
1. Ingresar alumnos y paradas con geolocalización
- ↓
2. Sistema calcula ruta óptima automáticamente (algoritmo TSP)
- ↓
3. Asigna ruta al bus y al piloto
- ↓
4. Piloto/Monitor acceden a su ruta desde su panel
- ↓
5. Ven mapa con orden de paradas y horarios estimados

Beneficios:

- Rutas optimizadas (ahorro 20-30% combustible)
- Cambios instantáneos (todos actualizados)
- Visibilidad total



Flujo: Registro de Recogidas

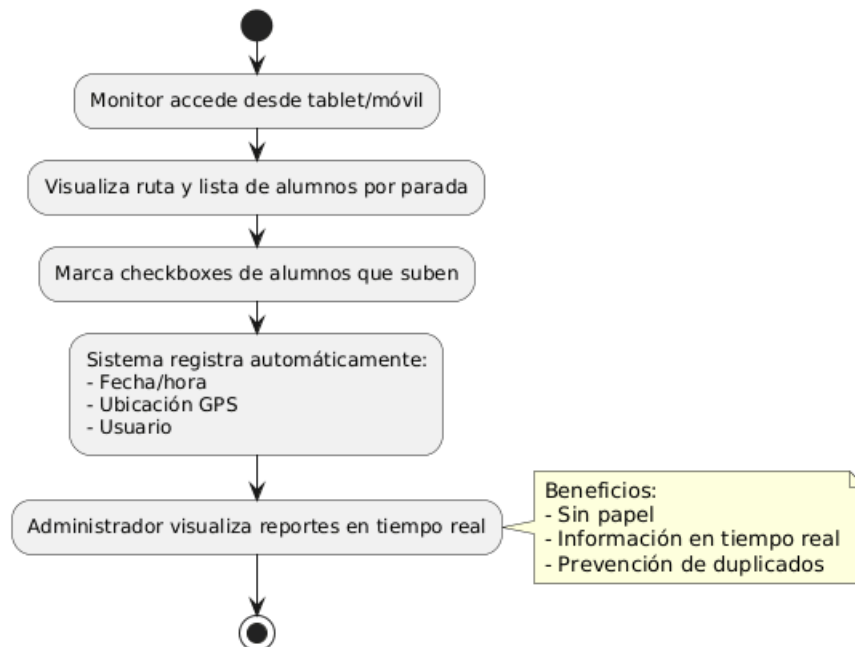
[Monitor en el sistema (tablet/móvil)]

- ↓
1. Ve su ruta con lista de alumnos por parada
- ↓
2. En cada parada, marca checkboxes de quién subió
- ↓
3. Sistema registra automáticamente:
 - Fecha/hora
 - Ubicación GPS
 - Usuario que registró
- ↓
4. Administrador ve reportes en tiempo real

Beneficios:

- Digitalizado, sin papel
- Información en tiempo real
- Prevención de duplicados

Proceso Automatizado - Registro de Recogidas



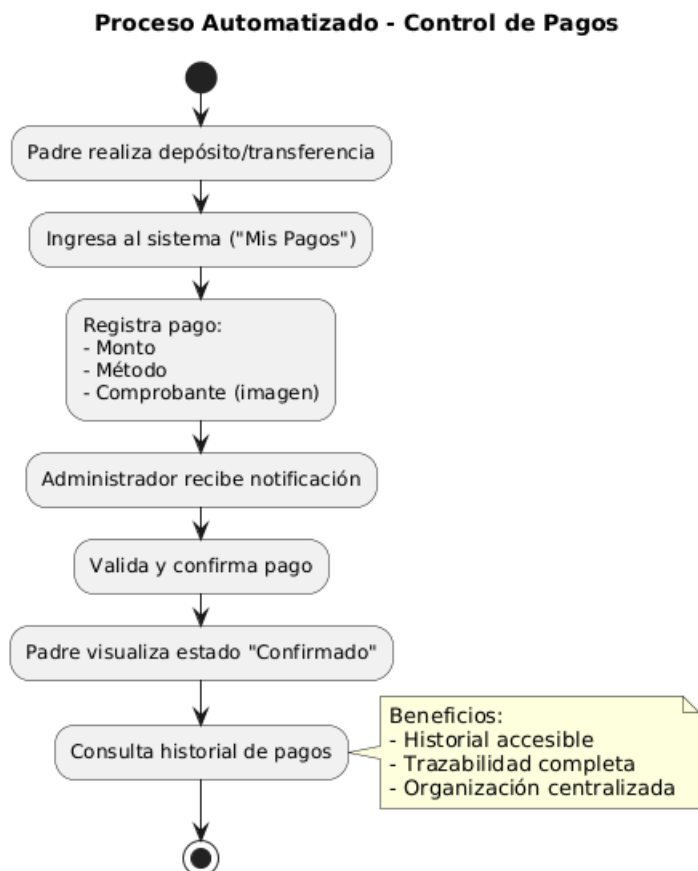
Flujo: Control de Pagos

[Padre de Familia en el sistema]

- ↓
1. Hace depósito/transferencia bancaria
- ↓
2. Entra al sistema, sección "Mis Pagos"
- ↓
3. Ingresa:
 - Monto
 - Método (transferencia/efectivo)
 - Comprobante (sube foto)
- ↓
4. Administrador recibe notificación, valida y confirma
- ↓
5. Padre ve estado actualizado: "Confirmado"
- ↓
6. Historial de pagos siempre accesible

Beneficios:

- Padre tiene acceso a su historial
- Trazabilidad completa
- Organización centralizada (no WhatsApp)



1.3 Análisis de Requerimientos Funcionales

1.3.1 Requerimientos Funcionales por Módulo

RF-01: Módulo de Autenticación y Autorización

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-01.1	El sistema debe permitir login con email y contraseña	 Crítica	Todos	Usuario accede con credenciales válidas
RF-01.2	El sistema debe soportar 4 roles: Administrador, Piloto, Monitor, Padre de Familia	 Crítica	Sistema	Cada usuario solo ve funcionalidades de su rol
RF-01.3	El sistema debe forzar cambio de contraseña en primer login	 Alta	Todos	Usuario no puede acceder hasta cambiar contraseña
RF-01.4	El sistema debe permitir logout	 Crítica	Todos	Sesión se cierra correctamente
RF-01.5	El sistema debe redirigir a cada rol a su página principal después del login	 Alta	Sistema	Administrador → Admin panel, Piloto → MiRuta, Monitor → MiRuta, Padre → Mapa
RF-01.6	El sistema debe mostrar mensaje de error si credenciales son inválidas	 Alta	Todos	Mensaje: "Email o contraseña incorrectos"

RF-02: Módulo de Gestión de Alumnos

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-02.1	El administrador debe poder crear un alumno con: nombre, apellido, fecha de nacimiento, grado	● Crítica	Administrador	Alumno se guarda en BD
RF-02.2	El administrador debe poder asignar un alumno a un padre de familia	● Crítica	Administrador	Relación en tabla AlumnoPadreFamilia
RF-02.3	El administrador debe poder asignar un alumno a una parada	● Crítica	Administrador	Registro en AsignacionAlumnoParada
RF-02.4	El administrador debe poder listar todos los alumnos	● Crítica	Administrador	Tabla con alumnos y acciones
RF-02.5	El administrador debe poder editar un alumno	● Alta	Administrador	Cambios se reflejan en BD
RF-02.6	El administrador debe poder eliminar un alumno (soft delete)	● Alta	Administrador	Alumno no se muestra, pero se mantiene en BD
RF-02.7	El sistema debe validar que la fecha de nacimiento sea coherente (alumno entre 3-12 años)	● Media	Sistema	Error si fecha no es válida

RF-03: Módulo de Gestión de Paradas

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-03.1	El administrador debe poder administrar orden de parada	● Crítica	Administrador	Parada se guarda en BD
RF-03.2	El sistema debe mostrar la ubicación en un mapa interactivo	● Alta	Administrador	El mapa rellena latitud/longitud automáticamente
RF-03.3	El administrador debe poder listar todas las paradas	● Crítica	Administrador	Tabla con paradas y acciones
RF-03.4	El sistema debe validar que latitud esté entre -90 y 90	● Alta	Sistema	Error si latitud inválida
RF-03.5	El sistema debe validar que longitud esté entre -180 y 180	● Alta	Sistema	Error si longitud inválida

RF-04: Módulo de Cálculo de Rutas

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-04.1	El administrador debe poder calcular una ruta a través del sistema que lo automáticamente para un bus y turno específico	● Crítica	Administrador	Sistema ejecuta algoritmo TSP y genera ruta óptima
RF-04.2	El sistema debe calcular rutas diferenciadas para turno Mañana y Tarde	● Crítica	Sistema	Dos rutas independientes por bus

RF-04.3	El sistema debe asignar un orden secuencial a cada parada en la ruta	 Crítica	Sistema	Campo Orden en ParadaRuta
RF-04.4	El sistema debe calcular horario estimado para cada parada	 Alta	Sistema	Campo HorarioEstimado basado en distancia y velocidad promedio
RF-04.5	El sistema debe permitir al administrador editar manualmente el orden de paradas	 Media	Administrador	Drag-and-drop o campo numérico editable
RF-04.6	El sistema debe recalcular la ruta si se agregan o eliminan paradas	 Media	Sistema	Ruta se actualiza automáticamente
RF-04.7	El sistema debe mostrar la ruta calculada en un mapa	 Alta	Administrador	Mapa con línea que conecta paradas en orden

RF-05: Módulo de Geolocalización en Tiempo Real

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-05.1	El piloto debe poder enviar su ubicación GPS en tiempo real	 Crítica	Piloto	Ubicación se envía vía SignalR cada 10 segundos
RF-05.2	El padre de familia debe poder ver el bus en un mapa en tiempo real	 Crítica	Padre	Mapa muestra ícono del bus en posición actual

RF-05.3	El sistema debe mostrar la ruta completa con paradas en el mapa	● Alta	Padre, Piloto	Mapa con marcadores en cada parada
RF-05.4	El sistema debe actualizar la posición del bus sin recargar la página	● Crítica	Sistema	SignalR actualiza mapa dinámicamente
RF-05.5	El sistema debe mostrar mensaje si el bus no está transmitiendo ubicación	● Alta	Padre	Mensaje: “El bus no está en operación actualmente”
RF-05.6	El sistema debe registrar historial de ubicaciones (opcional)	● Baja	Sistema	Tabla Historial Ubicaciones (futuro)

RF-06: Módulo de Pagos

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-06.1	El padre debe poder registrar un pago con: monto, fecha, método, comprobante o pago en línea	● Crítica	Padre	Pago se guarda con estado “Pendiente”
RF-06.2	El padre debe poder subir una imagen del comprobante de pago	● Alta	Padre	Imagen se guarda en servidor o Azure Blob

RF-06.3	El padre debe poder ver su historial de pagos	● Crítica	Padre	Tabla con todos sus pagos y estados
RF-06.4	El administrador debe poder listar todos los pagos pendientes	● Crítica	Administrador	Tabla filtrada por estado "Pendiente"
RF-06.5	El administrador debe poder confirmar un pago	● Crítica	Administrador	Estado cambia a "Confirmado"
RF-06.6	El administrador debe poder rechazar un pago con motivo	● Alta	Administrador	Estado cambia a "Rechazado", se registra motivo
RF-06.7	El sistema debe permitir filtrar pagos por alumno, fecha, estado	● Media	Administrador	Filtros funcionales en listado
RF-06.8	El sistema debe calcular el total de ingresos en un rango de fechas	● Media	Administrador	Reporte con suma de pagos confirmados

RF-07: Módulo de Confirmación de Asistencia

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-07.1	El padre debe poder confirmar diariamente si su hijo asistirá	● Crítica	Padre	Registro en tabla ConfirmacionAsistencia
RF-07.2	El sistema debe mostrar el estado de confirmación para el día actual	● Alta	Padre	Indicador visual: "Confirmado" / "No confirmado"
RF-07.3	El piloto/monitor debe poder ver qué alumnos confirmaron asistencia	● Crítica	Piloto, Monitor	Lista con checkmarks de confirmaciones
RF-07.4	El sistema debe permitir cambiar la confirmación hasta 1 hora antes del inicio de ruta	● Media	Padre	Después de la hora límite, no permite cambios
RF-07.5	El sistema debe enviar recordatorio si el padre no ha confirmado (futuro)	● Baja	Sistema	Notificación o email automático

RF-08: Módulo de Registro de Recogidas (Monitor)

ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-08.1	El monitor debe poder ver su ruta diaria con lista de alumnos por parada	● Crítica	Monitor	Página RegistrarRecogidas con ruta y alumnos
RF-08.2	El monitor debe poder marcar qué alumnos fueron recogidos en cada parada	● Crítica	Monitor	Checkboxes por alumno, botón "Guardar"
RF-08.3	El sistema debe registrar fecha/hora y ubicación GPS al guardar recogidas	● Alta	Sistema	Campos automáticos en RegistroRecogida
RF-08.4	El sistema debe prevenir registros duplicados del mismo alumno en el mismo día	● Crítica	Sistema	Validación: error si ya existe registro
RF-08.5	El monitor debe poder ver qué alumnos ya fueron registrados	● Alta	Monitor	Checkboxes pre-marcados si ya hay registro
RF-08.6	El administrador debe poder ver reportes de recogidas por fecha y bus	● Alta	Administrador	Reporte con filtros

RF-9: Dashboard y Reportes

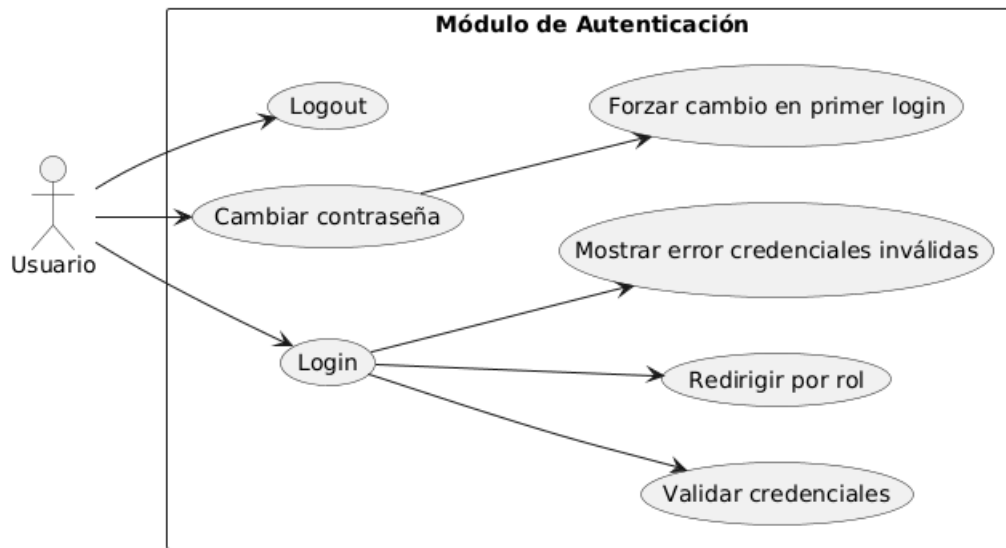
ID	Requerimiento	Prioridad	Actor	Criterio de Aceptación
RF-9.1	El administrador debe ver un dashboard con estadísticas generales	● Alta	Administrador	Tarjetas con: total de pagos pendientes.
RF-9.2	El administrador debe poder generar reporte de pagos por rango de fechas	● Alta	Administrador	Detalle de pagos

1.3.2 Matriz de Trazabilidad (Requerimientos ↔ Casos de Uso)

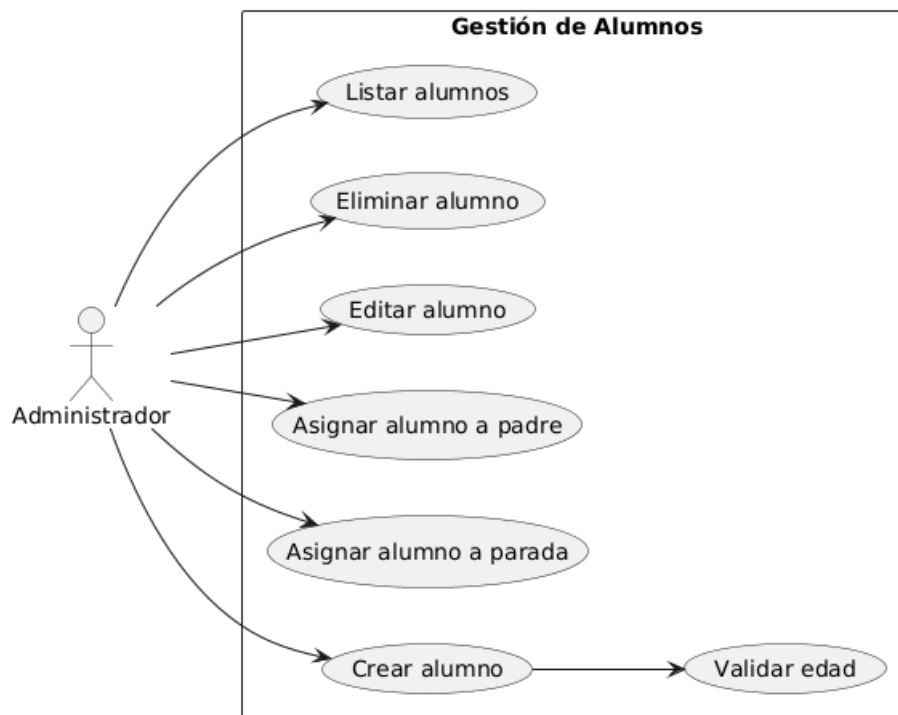
Caso de Uso	Requerimientos Relacionados	Prioridad	Complejidad
CU-01: Login	RF-01.1, RF-01.2, RF-01.5, RF-01.6	● Crítica	● Baja
CU-02: Cambiar contraseña	RF-01.3	● Alta	● Baja
CU-05: Crear alumno	RF-02.1, RF-02.7	● Crítica	● Baja
CU-06: Asignar alumno a parada	RF-02.3	● Crítica	● Media
CU-07: Crear parada	RF-03.1, RF-03.6, RF-03.7	● Crítica	● Media
CU-08: Seleccionar ubicación en mapa	RF-03.2	● Alta	● Media
CU-09: Calcular ruta automáticamente	RF-04.1, RF-04.2, RF-04.3, RF-04.4	● Crítica	● Alta
CU-10: Ver ruta en mapa	RF-04.7	● Alta	● Media

CU-11: Enviar ubicación GPS (piloto)	RF-05.1	● Crítica	● Alta
CU-12: Ver bus en tiempo real (padre)	RF-05.2, RF-05.3, RF-05.4	● Crítica	● Alta
CU-13: Registrar pago	RF-06.1, RF-06.2	● Crítica	● Media
CU-14: Confirmar pago	RF-06.5	● Crítica	● Baja
CU-15: Ver historial de pagos	RF-06.3	● Crítica	● Baja
CU-16: Confirmar asistencia	RF-07.1, RF-07.2	● Crítica	● Baja
CU-17: Registrar recogidas	RF-08.1, RF-08.2, RF-08.3, RF-08.4	● Crítica	● Media
CU-18: Ver dashboard	RF-9.1	● Alta	● Media
CU-19: Generar reporte de pagos	RF-9.2	● Alta	● Media

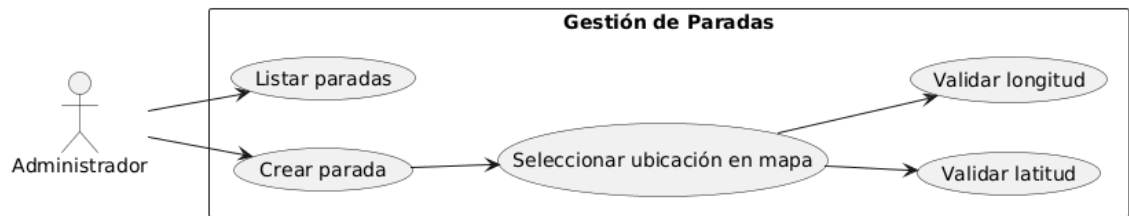
- Autenticación (CU-01, CU-02)



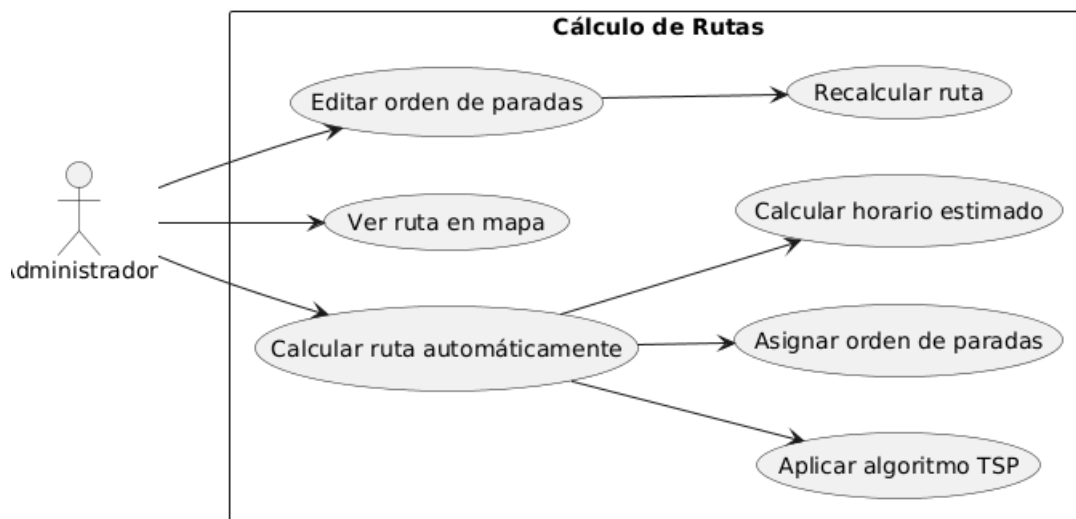
- Gestión de Alumnos (CU-05, CU-06)



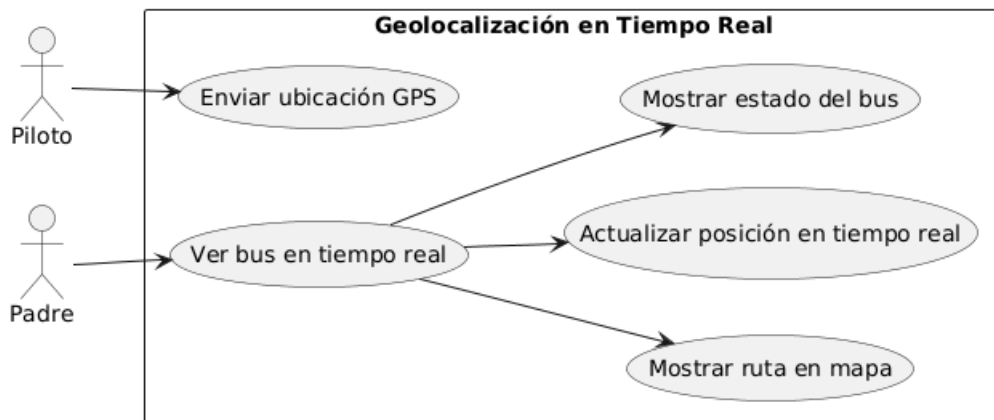
- **Gestión de Paradas (CU-07, CU-08)**



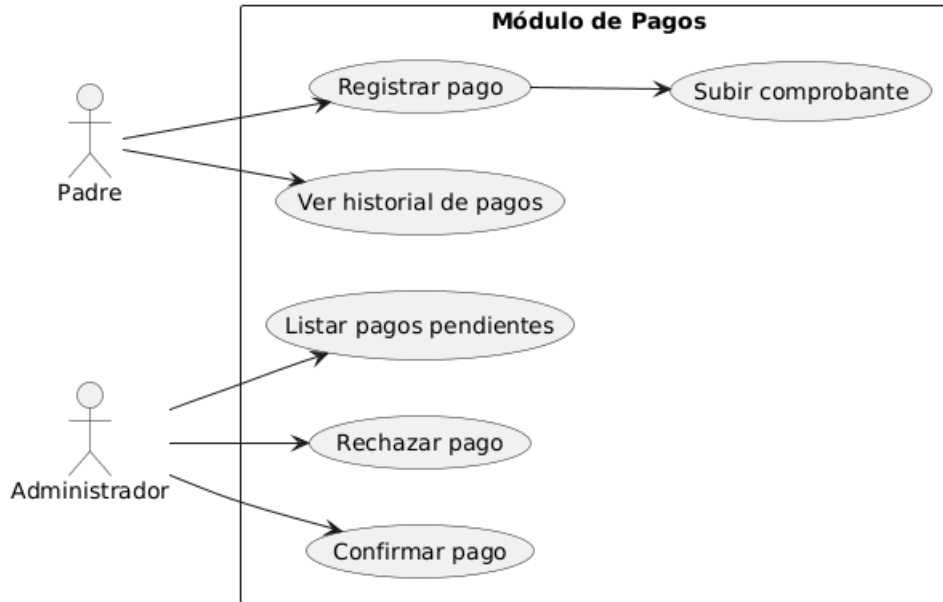
- **Rutas (CU-09, CU-10)**



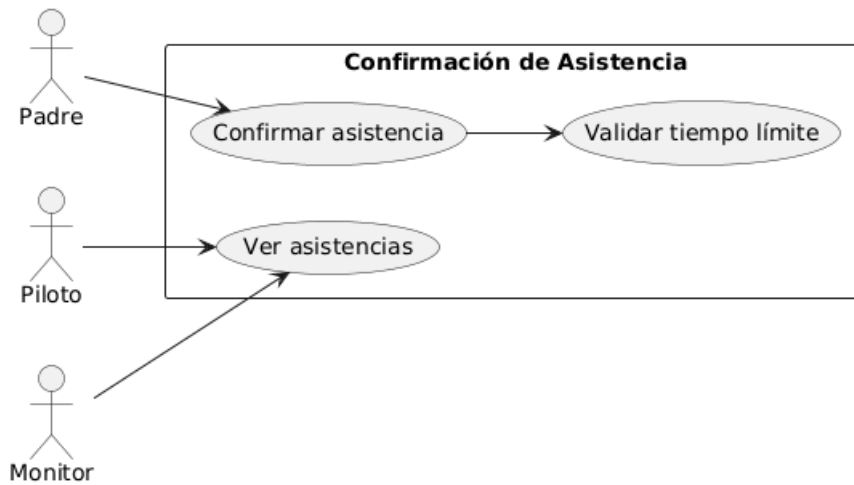
- **Geolocalización (CU-11, CU-12)**



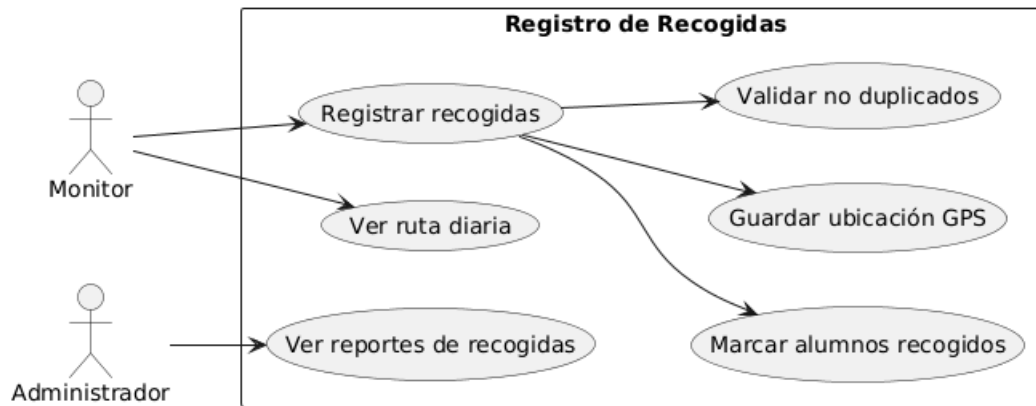
- Pagos (CU-13, CU-14, CU-15)



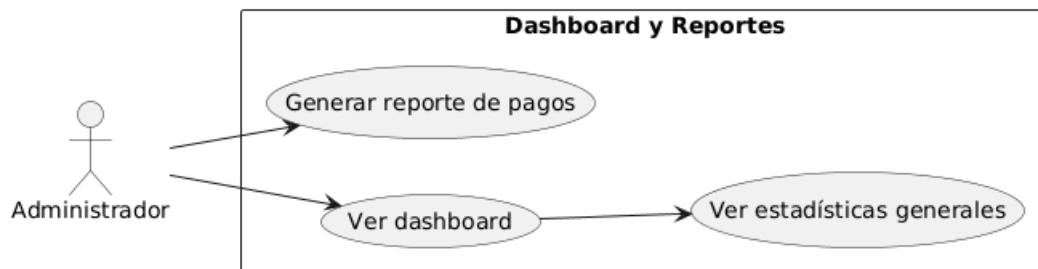
- Asistencia (CU-16)



- **Recogidas (CU-17)**



- **Dashboard y Reportes (CU-18, CU-19)**



1.4 Análisis de Requerimientos No Funcionales

1.4.1 Requerimientos de Rendimiento (Performance)

ID	Requerimiento	Métrica	Prioridad	Justificación
RNF-01.1	El tiempo de carga de la página principal no debe exceder 2 segundos	< 2s	● Alta	Experiencia de usuario fluida
RNF-01.2	El cálculo de una ruta con hasta 30 paradas no debe tomar más de 5 segundos	< 5s	● Alta	Evitar frustración del administrador
RNF-01.3	El sistema debe soportar hasta 100 usuarios concurrentes sin degradación	100 usuarios	● Alta	Empresas medianas con muchos padres
RNF-01.4	La actualización de ubicación GPS debe ocurrir cada 10 segundos máximo	≤ 10s	● Crítica	Geolocalización en "tiempo real"
RNF-01.5	Las consultas a base de datos deben responder en menos de 500ms	< 500ms	● Alta	Fluidez en operaciones CRUD
RNF-01.6	El sistema debe soportar hasta 50 buses transmitiendo ubicación simultáneamente	50 buses	● Media	Escalabilidad para empresas grandes

1.4.2 Requerimientos de Usabilidad

ID	Requerimiento	Criterio de Aceptación	Prioridad	Target
RNF-02.1	La interfaz debe ser responsive y funcionar en móviles, tablets y desktops	Bootstrap 5 responsive	● Crítica	Padres usan móviles
RNF-02.2	El sistema debe ser intuitivo: un usuario promedio debe poder usarlo sin capacitación extensa	Usuario completa tarea en < 5 min	● Alta	Pilotos/monitores no son expertos en tecnología
RNF-02.3	Los mensajes de error deben ser claros y en español	Mensajes descriptivos (no códigos técnicos)	● Alta	Evitar confusión
RNF-02.4	Los botones y enlaces deben tener tamaños adecuados para touch	Guías de accesibilidad móvil	● Alta	Usabilidad en tablets para monitores
RNF-02.5	El sistema debe mostrar loaders/spinner s mientras procesa operaciones largas	Indicador visual de carga	● Alta	Feedback al usuario
RNF-02.6	Los formularios deben tener validaciones en tiempo real	Validaciones client-side con JavaScript	● Media	Mejor UX

1.4.3 Requerimientos de Seguridad

ID	Requerimiento	Método de Implementación	Prioridad	Riesgo Mitigado
RNF-03.1	Todas las contraseñas deben ser hasheadas (nunca plain text)	ASP.NET Core Identity (bcrypt/PBKDF2)	 Crítica	Robo de credenciales
RNF-03.2	Las conexiones deben ser HTTPS (certificado SSL)	SSL/TLS en Azure	 Crítica	Interceptación de datos (MITM)
RNF-03.3	Las sesiones deben expirar después de 30 minutos de inactividad	Configuración de cookies de sesión	 Alta	Sesiones abiertas en computadoras compartidas
RNF-03.4	Los archivos subidos (comprobantes) deben validarse (tipo, tamaño)	Validación: JPEG/PNG, máx 5MB	 Alta	Subida de archivos maliciosos
RNF-03.5	El sistema debe prevenir SQL Injection	Entity Framework Core (ORM)	 Crítica	Ataques de inyección
RNF-03.6	El sistema debe prevenir XSS (Cross-Site Scripting)	Razor Pages sanitiza inputs automáticamente	 Crítica	Inyección de scripts maliciosos
RNF-03.7	El acceso a funcionalidades debe estar restringido por rol	[Authorize(Roles = "...")] en cada página	 Crítica	Acceso no autorizado

1.4.4 Requerimientos de Disponibilidad (Availability)

ID	Requerimiento	Métrica	Prioridad	SLA
RNF-04.1	El sistema debe estar disponible 99.5% del tiempo mensual	Uptime $\geq$ 99.5%	● Alta	~3.6 horas de downtime permitido/mes
RNF-04.2	Las actualizaciones de mantenimiento deben hacerse en horarios de baja actividad	Domingos 2-5 AM	● Media	Minimizar impacto en usuarios
RNF-04.3	El sistema debe tener backups automáticos diarios de la base de datos	Backup diario a las 3 AM	● Crítica	Recuperación ante desastres
RNF-04.4	Los backups deben retenerse por al menos 30 días	Retention: 30 días	● Alta	Recuperación de datos históricos

1.4.5 Requerimientos de Escalabilidad

ID	Requerimiento	Criterio	Prioridad	Justificación
RNF-05.1	La base de datos debe poder almacenar al menos 1 año de historial de ubicaciones	365 días de registros GPS	● Media	Análisis histórico
RNF-05.2	La arquitectura debe permitir agregar nuevos módulos sin refactoring mayor	Arquitectura modular (áreas Razor Pages)	● Alta	Extensibilidad futura
RNF-05.3	El sistema debe permitir multi-tenancy (múltiples empresas en el futuro)	Diseño con IdEmpresa en tablas	● Baja (v2.0)	Escalabilidad del negocio

1.4.6 Requerimientos de Mantenibilidad

ID	Requerimiento	Método	Prioridad	Beneficio
RNF-06.1	El código debe seguir convenciones de C# y Razor Pages	Estándares de Microsoft	● Alta	Legibilidad y mantenimiento
RNF-06.2	Las funciones complejas deben estar documentadas con comentarios XML	/// <summary> en métodos públicos	● Media	Facilitar onboarding de nuevos devs
RNF-06.3	El sistema debe tener logging de errores centralizado	Serilog o Azure Application Insights	● Alta	Debugging y monitoreo
RNF-06.4	Los cambios en base de datos deben gestionarse con migraciones de EF Core	dotnet ef migrations add	● Crítica	Control de versiones de BD

1.4.7 Requerimientos de Compatibilidad

ID	Requerimiento	Detalle	Prioridad
RNF-07.1	El sistema debe funcionar en navegadores modernos: Chrome, Firefox, Edge, Safari	Últimas 2 versiones de cada navegador	● Crítica
RNF-07.2	El sistema debe funcionar en dispositivos Android e iOS (vía navegador)	Responsive web, no app nativa	● Crítica
RNF-07.3	El sistema debe ser compatible con resoluciones desde 320px (móviles) hasta 1920px (desktops)	Bootstrap breakpoints	● Alta
RNF-07.4	El sistema debe funcionar con conexiones de internet moderadas (3G mínimo)	Optimización de recursos, caching	● Media

1.4.8 Requerimientos de Cumplimiento Legal

ID	Requerimiento	Normativa	Prioridad	Acción Requerida
RNF-08.1	El sistema debe cumplir con leyes de protección de datos personales (si aplican en Honduras)	GDPR (referencia), ley local	● Alta	Política de privacidad, consentimiento de padres
RNF-08.2	Los datos de menores (alumnos) deben manejarse con protección especial	COPPA (referencia)	● Alta	No recopilar datos innecesarios de alumnos
RNF-08.3	El sistema debe permitir a los usuarios exportar o eliminar sus datos (derecho al olvido)	GDPR Art. 17	● Media	Funcionalidad "Exportar mis datos" / "Eliminar mi cuenta"
RNF-08.4	El sistema debe tener términos y condiciones y política de privacidad	Legal	● Alta	Páginas estáticas con documentos legales

1.5 Análisis de Dominio

1.5.1 Modelo Conceptual del Negocio

[DIAGRAMA DE DOMINIO]

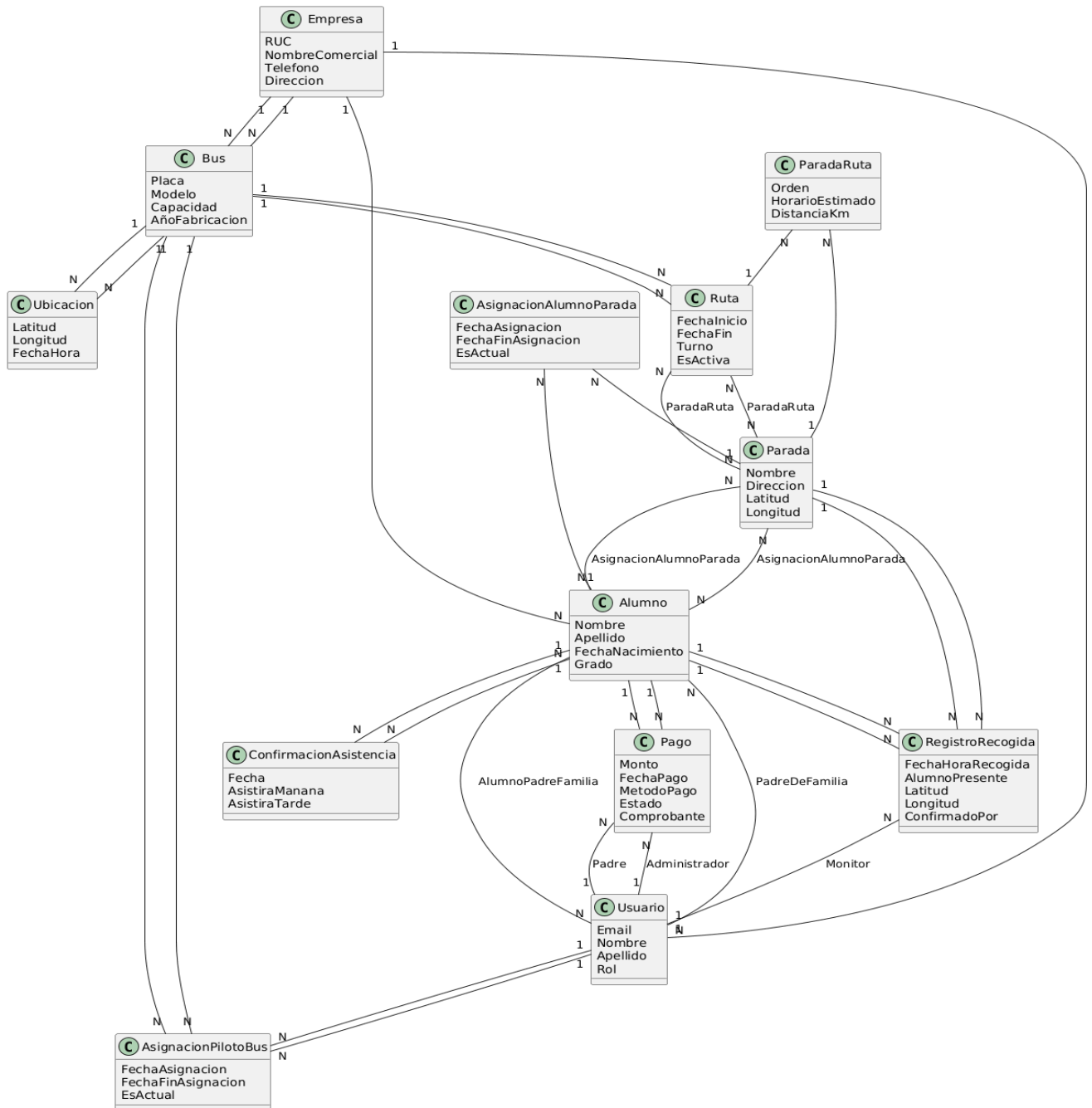
Título: Modelo de Dominio - Transportes Genesis

Entidades Principales y Relaciones:

1. Empresa (entidad raíz)
 - Atributos: RUC, NombreComercial, Teléfono, Dirección
 - Relaciones:
 - * Tiene muchos Buses (1:N)
 - * Tiene muchos Usuarios (1:N)
 - * Tiene muchos Alumnos (1:N)
2. Bus
 - Atributos: Placa (único), Modelo, Capacidad, AñoFabricación
 - Relaciones:
 - * Pertenece a una Empresa (N:1)
 - * Tiene AsignacionesPilotoBus (1:N)
 - * Tiene Rutas (1:N por turno)
 - * Transmite Ubicaciones (1:N)
3. Usuario (AppUser)
 - Atributos: Email, Nombre, Apellido, Rol (Administrador, Piloto, Monitor, PadreDeFamilia)
 - Relaciones:
 - * Si es Piloto/Monitor → Tiene AsignacionesPilotoBus (1:N)
 - * Si es Padre → Tiene Alumnos (1:N)
4. Alumno
 - Atributos: Nombre, Apellido, FechaNacimiento, Grado
 - Relaciones:
 - * Pertenece a Padres de Familia (N:N) vía AlumnoPadreFamilia
 - * Asignado a Paradas (N:N) vía AsignacionAlumnoParada
 - * Tiene ConfirmacionesAsistencia (1:N)
 - * Tiene RegistrosRecogida (1:N)
 - * Tiene Pagos asociados (1:N)
5. Parada
 - Atributos: Nombre, Dirección, Latitud, Longitud
 - Relaciones:
 - * Pertenece a Rutas (N:N) vía ParadaRuta
 - * Tiene Alumnos asignados (N:N) vía AsignacionAlumnoParada
 - * Tiene RegistrosRecogida (1:N)

6. Ruta
 - Atributos: FechaInicio, FechaFin, Turno (Mañana/Tarde), EsActiva
 - Relaciones:
 - * Asignada a un Bus (N:1)
 - * Tiene Paradas (N:N) vía ParadaRuta con atributo Orden
7. ParadaRuta (tabla intermedia con datos)
 - Atributos: Orden, HorarioEstimado, DistanciaKm
 - Relaciona: Ruta ↔ Parada
8. AsignacionPilotoBus (tabla de relación histórica)
 - Atributos: FechaAsignacion, FechaFinAsignacion, EsActual
 - Relaciona: Usuario (Piloto/Monitor) ↔ Bus
9. AsignacionAlumnoParada (tabla de relación histórica)
 - Atributos: FechaAsignacion, FechaFinAsignacion, EsActual
 - Relaciona: Alumno ↔ Parada
10. ConfirmacionAsistencia
 - Atributos: Fecha, AsistiraManana, AsistiraTarde
 - Relaciona: Alumno + Fecha (confirmación diaria)
11. RegistroRecogida
 - Atributos: FechaHoraRecogida, AlumnoPresente, Latitud, Longitud, ConfirmadoPor
 - Relaciona: Alumno + Parada + Fecha + Usuario (Monitor)
12. Pago
 - Atributos: Monto, FechaPago, MetodoPago, Estado, Comprobante
 - Relaciona: Alumno + Usuario (Padre) + Usuario (Administrador que confirma)
13. Ubicacion (transmisión GPS en tiempo real)
 - Atributos: Latitud, Longitud, FechaHora
 - Relaciona: Bus (transmisión continua vía SignalR)

Modelo de Dominio - Transportes Genesis



Conceptos de Dominio (Glosario):

Turno: Periodo del día (Mañana o Tarde) en que opera una ruta

Ruta Activa: Ruta vigente y en operación

Orden de Parada: Secuencia numérica en que se visitan las paradas en una ruta

TSP (Traveling Salesman Problem): Algoritmo que calcula la ruta más corta visitando todas las paradas

Geolocalización: Ubicación geográfica expresada en latitud y longitud

SignalR: Tecnología de comunicación en tiempo real (WebSockets)

Rol: Perfil de usuario con permisos específicos

Soft Delete: Eliminación lógica (registro marcado como eliminado pero no borrado físicamente)

1.5.2 Reglas de Negocio

ID	Regla de Negocio	Justificación	Implementación
RN-01	Un bus solo puede tener un piloto activo a la vez	Evitar confusión sobre quién opera el bus	Campo EsActual en AsignacionPilotoBus
RN-02	Un alumno solo puede estar asignado a una parada activa a la vez	Un alumno solo puede ser recogido en una ubicación por turno	Campo EsActual en AsignacionAlumnoParada
RN-03	Un bus solo puede tener una ruta activa por turno (Mañana y Tarde)	Evitar ambigüedad sobre qué ruta seguir	Campo Turno + EsActiva en Ruta
RN-04	Una confirmación de asistencia solo es válida para el día actual	Confirmaciones son diarias y no se arrastran	Campo Fecha en ConfirmacionAsistencia
RN-05	No se puede registrar la misma recogida dos veces (mismo alumno, misma parada, mismo día)	Evitar duplicados	Validación en controller/service
RN-06	Un pago en estado "Pendiente" puede ser editado por el padre; uno "Confirmado" no	Integridad de registros contables	Validación de estado antes de editar

RN-07	La fecha de nacimiento de un alumno debe estar en el rango de 3-18 años	Alumnos de transporte escolar son niños/adolescentes	Validación en modelo
RN-08	La placa de un bus debe ser única en el sistema	Identificación inequívoca de buses	Índice único en BD + validación
RN-09	Un usuario con rol "Piloto" o "Monitor" debe estar asignado a un bus para acceder a su ruta	Sin asignación, no hay ruta que mostrar	Validación en página MiRuta.cshtml.cs
RN-10	El cálculo de ruta debe considerar solo paradas con alumnos asignados	No tiene sentido visitar paradas vacías	Filtro en servicio de cálculo de rutas
RN-11	Los horarios estimados de paradas se calculan asumiendo velocidad promedio de 30 km/h	Estimación realista para zonas urbanas	Fórmula: distancia / velocidad
RN-12	Un padre solo puede ver información de sus propios hijos	Privacidad y seguridad	Filtro por IdUsuarioPadre en consultas
RN-13	Al eliminar un bus (soft delete), las rutas asociadas se marcan como inactivas	Consistencia de datos	Lógica en servicio de eliminación
RN-14	La ubicación GPS debe actualizarse máximo cada 10 segundos para considerarse "tiempo real"	Balance entre precisión y consumo de datos	Timer en cliente SignalR
RN-15	Solo el administrador puede confirmar o rechazar pagos	Control financiero centralizado	[Authorize(Roles = "Administrador")]

1.5.3 Procesos de Negocio Clave

Proceso 1: Onboarding de Nuevo Alumno

[Administrador]

- ↓
- 1. Crea usuario "Padre de Familia" (email + contraseña temporal)
- ↓
- 3. Vincula Alumno con Padre (tabla AlumnoPadreFamilia)
- ↓
- 4. Asigna Alumno a una Parada existente (tabla AsignacionAlumnoParada, Es Actual = true)
- ↓
- 5. Recalcula rutas afectadas (servicio TSP)
- ↓
- 6. Notifica a Padre (por correo o WhatsApp) con credenciales de acceso
- ↓
- [Padre recibe email, accede al sistema, cambia contraseña obligatoriamente]

Proceso 2: Operación Diaria de Ruta (Turno Mañana)

[Padre de Familia]

- ↓
- Confirma asistencia de su hijo para mañana (página "Confirmar Asistencia")
- ↓
- Sistema registra en `ConfirmacionAsistencia` (Fecha = mañana, AsistiraManana = true)

[Piloto]

- ↓
- 1. Inicia sesión, ve su ruta diaria en `MiRuta`
- ↓
- 2. Revisa lista de alumnos confirmados
- ↓
- 3. Inicia transmisión de ubicación GPS (activo botón "Iniciar Ruta")
- ↓
- 4. SignalR envía ubicación cada 10 segundos

[Monitor] (en paralelo)

- ↓
- 1. Abre página `RegistrarRecogidas` en tablet
- ↓
- 2. Ve ruta con paradas y alumnos
- ↓
- 3. En cada parada:
 - Marca checkboxes de alumnos que subieron al bus
 - Presiona "Guardar"

- Sistema registra en `RegistroRecogida` (FechaHora, Lat, Lon automáticos)

↓

4. Continúa hasta completar todas las paradas

[Padre de Familia] (en paralelo)

↓

1. Abre el sistema desde su móvil

↓

2. Ve mapa con bus en tiempo real

↓

3. Observa cuándo el bus está cerca de su parada

↓

4. Confirma que su hijo fue recogido (ve checkbox marcado en su panel)

[Administrador]

↓

Al final del día, revisa dashboard:

- Alumnos confirmados vs recogidos
- Rutas completadas
- Incidencias (alumnos confirmados, pero no recogidos)

Proceso 3: Ciclo de Pago Mensual

Inicio de Mes:

[Administrador]

↓

Genera factura/recibo mensual para cada alumno (puede ser manual o futura feature)

↓

Envía por email o WhatsApp a padres

Durante el Mes:

[Padre de Familia]

↓

1. Hace depósito bancario o transferencia

↓

2. Accede al sistema, sección "Mis Pagos"

↓

3. Clic en "Registrar Pago"

↓

4. Completo formulario:

- Alumno
- Monto
- Fecha de pago
- Método (Transferencia/Efectivo/Tarjeta)
- Sube imagen del comprobante

↓

5. Presiona "Guardar"

↓
Sistema guarda pago con Estado = "Pendiente"

Validación:

[Administrador]

- ↓
1. Ve listado de "Pagos Pendientes"
 - ↓
 2. Revisa comprobante subido
 - ↓
 3. Valida contra registros bancarios
 - ↓
 4. Opciones:
 - a) Confirmar pago → Estado = "Confirmado"
 - b) Rechazar pago → Estado = "Rechazado", ingresa motivo ("Monto incorrecto", "Comprobante ilegible", etc.)

↓
Sistema actualiza estado

Consulta:

[Padre de Familia]

↓
Entra a "Historial de Pagos"

- ↓
- Ve tabla con todos sus pagos:
- Fecha
 - Monto
 - Estado (Pendiente / Confirmado / Rechazado)
 - Motivo (si fue rechazado)

1.6 Supuestos, restricciones y dependencias funcionales

1.6.1 Supuestos del Proyecto

ID	Supuesto	Implicación	Riesgo si es Falso
SUP-01	Los usuarios (pilotos, monitores, padres) tienen acceso a smartphones o computadoras con navegador web	El sistema es 100% web; no hay app nativa	● Medio: Si muchos usuarios no tienen dispositivos, adopción será baja
SUP-02	Los usuarios tienen conexión a internet (3G mínimo) durante la operación	Geolocalización y actualización en tiempo real dependen de conectividad	● Alto: Sin internet, features críticas no funcionan
SUP-03	Los navegadores de los usuarios son modernos (últimas 2 versiones de Chrome, Firefox, Edge, Safari)	Se usan features modernas de HTML5, CSS3, JavaScript	● Bajo: Navegadores antiguos son minoría
SUP-04	La empresa de transporte ya tiene definidas sus paradas y rutas operativas	El sistema digitaliza rutas existentes	● Medio: Si no hay rutas establecidas, requiere trabajo previo de campo
SUP-05	Los pilotos y monitores tienen dispositivos (tablets o móviles) para usar durante la operación	Registro de recogidas y envío de GPS ocurren en movimiento	● Medio: Si no tienen dispositivos, la empresa debe proveerlos
SUP-06	Los padres están dispuestos a usar una plataforma digital	Cambio cultural de WhatsApp/llamadas a sistema web	● Medio: Resistencia al cambio puede frenar adopción

SUP-07	Los datos de ubicación GPS de los dispositivos son precisos (margen de error ≤ 10 metros)	Mapas en tiempo real muestran posición real del bus	● Bajo: GPS moderno es generalmente preciso
SUP-8	La empresa tiene personal (administrador) con conocimientos básicos de computación	Gestión del sistema requiere uso de interfaces CRUD	● Medio: Si el administrador no sabe usar computadoras, requiere capacitación intensiva

1.6.2 Restricciones del Proyecto

Restricciones Técnicas

ID	Restricción	Tipo	Impacto
RES-TEC-01	El sistema DEBE desarrollarse en .NET 8 con Razor Pages	Tecnología	No se puede usar Blazor, Angular, React, etc.
RES-TEC-02	La base de datos DEBE ser SQL Server	Infraestructura	No se puede usar MySQL, PostgreSQL, MongoDB, etc.
RES-TEC-03	El sistema DEBE desplegarse en Microsoft Azure	Infraestructura	No se puede usar AWS, Google Cloud, DigitalOcean, etc.
RES-TEC-04	La geolocalización DEBE usar Google Maps API	API Externa	No se puede usar OpenStreetMap, Mapbox (al menos en v1.0)
RES-TEC-05	El tiempo real DEBE implementarse con SignalR	Protocolo	No se puede usar polling, long-polling, otros frameworks WebSocket
RES-TEC-06	El sistema DEBE ser una aplicación web (NO app móvil nativa)	Plataforma	No hay versiones iOS/Android nativas en v1.0

Restricciones de Presupuesto

ID	Restricción	Límite	Impacto
RES-PRE-01	Inversión inicial máxima: Q16,000 USD	Desarrollo + capital de trabajo	No se pueden contratar más de 2-3 desarrolladores o extender timeline excesivamente
RES-PRE-02	Costos operativos mensuales máximos: Q2000 USD	Azure + Google Maps + otros	Debe optimizarse el uso de recursos cloud
RES-PRE-03	Precio de venta por cliente: Q 60/bus/mes	Modelo de negocio	No se puede cobrar más sin justificación de valor

Restricciones de Tiempo

ID	Restricción	Plazo	Impacto
RES-TIE-01	El MVP debe estar listo en 3 meses (12 semanas)	537 horas de desarrollo	Features no críticas deben diferirse a v2.0
RES-TIE-02	El sistema debe estar operativo para el inicio del ciclo escolar	Típicamente Febrero en Honduras	Deadline fijo; retrasos pueden perder oportunidad de mercado

Restricciones Legales y de Cumplimiento

ID	Restricción	Normativa	Impacto
RES-LEG-01	El sistema debe cumplir con leyes de protección de datos personales	Ley hondureña (si existe) o GDPR como referencia	Debe incluirse política de privacidad y consentimiento
RES-LEG-02	Los datos de menores (alumnos) deben manejarse con protección especial	COPPA (referencia)	No recopilar datos sensibles innecesarios
RES-LEG-03	El sistema debe tener términos y condiciones	Legal	Página estática obligatoria

1.6.3 Dependencias Funcionales

Dependencias Externas (Third-Party)

Dependencia	Proveedor	Criticidad	Función	Riesgo de Disponibilidad	Mitigación
Google Maps API	Google	● Crítica	Geolocalización, mapas, cálculo de distancias	● Baja (99.9% uptime)	Cache de mapas estáticos, fallback a OpenStreetMap en v2.0
Microsoft Azure	Microsoft	● Crítica	Hosting, base de datos, storage	● Baja (SLA 99.95%)	Backups regulares, plan de recuperación ante desastres
SignalR	Microsoft	● Crítica	Tiempo real (WebSockets)	● Baja (parte de .NET)	Fallback a long-polling si WebSockets falla
Bootstrap 5	Open Source (CDN)	● Alta	UI responsive	● Baja (múltiples CDNs)	Servir Bootstrap desde servidor propio si CDN falla
jQuery	Open Source (CDN)	● Alta	Interactividad frontend	● Baja	Servir desde servidor propio

Dependencias Internas (Entre Módulos)

Módulo Dependiente	Módulo del que Depende	Tipo de Dependencia	Impacto si el Módulo Base Falla
Cálculo de Rutas	Gestión de Paradas + Gestión de Alumnos	Datos	No se puede calcular ruta sin paradas y alumnos
Geolocalización en Tiempo Real	Gestión de Buses + Gestión de Rutas	Datos	No se puede mostrar bus en mapa sin bus y ruta asignados

Registro de Recogidas	Gestión de Rutas + Gestión de Alumnos	Datos	Monitor no puede registrar recogidas sin ruta y alumnos
Confirmación de Asistencia	Gestión de Alumnos + Auth (Padres)	Datos + Auth	Padre no puede confirmar sin alumno vinculado
Gestión de Pagos	Gestión de Alumnos + Auth (Padres, Administrador)	Datos + Auth	No se puede registrar pago sin alumno y usuarios
Dashboard de Administrador	Todos los módulos	Datos agregados	Dashboard completo y sistema sigue operativo

1.6.4 Riesgos Funcionales

ID	Riesgo	Probabilidad	Impacto	Mitigación
RF-RIESGO-01	El algoritmo TSP no optimiza rutas adecuadamente para casos complejos (50+ paradas)	● Media	● Medio	Implementar versión simplificada (greedy algorithm) + permitir ajuste manual
RF-RIESGO-02	Los pilotos olvidan activar la transmisión de ubicación GPS	● Media	● Alto	Mostrar alerta prominente en su dashboard + notificación si no transmite en horario de ruta
RF-RIESGO-03	Los monitores registran recogidas incorrectamente (marcan alumnos que no subieron)	● Media	● Medio	Validación cruzada con confirmaciones de asistencia + revisión de administrador
RF-RIESGO-04	Los padres no confirman asistencia (sistema asume que todos asistirán)	● Alta	● Bajo	Configuración: "Por defecto, todos asisten si no se confirma lo contrario"
RF-RIESGO-05	La conexión a internet del bus es intermitente, ubicación GPS se	● Media	● Medio	Buffer de ubicaciones en cliente, enviar cuando reconecta + mostrar "última ubicación conocida" en mapa

	actualiza con retraso			
RF-RIESGO-06	Los padres suben comprobantes de pago falsos o editados	● Baja	● Alto	Validación manual por administrador + verificación cruzada con extractos bancarios
RF-RIESGO-07	El sistema se usa para un solo bus (no escala a múltiples buses)	● Baja	● Bajo	Arquitectura ya soporta múltiples buses; solo es cuestión de configuración

1.1.5 Usuarios Finales del Sistema

El sistema está diseñado para cuatro tipos principales de usuarios:

1. **Administradores** (Gerentes/Dueños de la empresa)
 - Gestionan usuarios, buses, rutas, alumnos
 - Calculan rutas optimizadas
 - Visualizan reportes y estadísticas
 - Supervisan operaciones en tiempo real
2. **Pilotos** (Conductores de buses)
 - Visualizan su ruta asignada del día
 - Ven la secuencia de paradas y horarios
 - Consultan información de alumnos en cada parada
 - Acceden al mapa de su ruta
3. **Monitores** (Asistentes en el bus)
 - Visualizan la ruta del bus asignado
 - Registran qué alumnos fueron recogidos en cada parada
 - Confirman presencia o ausencia de alumnos
 - Acceden al mapa en tiempo real
4. **Padres de Familia** (Clientes)
 - Confirman asistencia diaria de sus hijos
 - Visualizan el bus en tiempo real
 - Registran pagos realizados
 - Consultan historial de pagos y asistencias

1.1.6 Beneficios Esperados

Para la Empresa de Transporte:

- **Reducción de costos** operativos (combustible) mediante rutas optimizadas
- **Mejora en eficiencia** operativa (procesos automatizados)
- **Mayor control** y trazabilidad de operaciones
- **Reducción de errores** humanos en registros
- **Mejor imagen corporativa** (tecnología moderna)
- **Escalabilidad** del negocio (más fácil gestionar más buses)

Para los Pilotos y Monitores:

- **Claridad en rutas** a seguir cada día
- **Reducción de tiempo** en planificación manual
- **Facilidad de registro** de recogidas (digital vs papel)
- **Acceso desde cualquier dispositivo** con internet

Para los Padres de Familia:

- **Tranquilidad** al ver ubicación del bus en tiempo real
- **Mejor comunicación** con la empresa (confirmaciones, pagos)
- **Transparencia** en registros y pagos
- **Facilidad de uso** (interfaz web responsive)

ESPECIFICACIÓN TÉCNICA - TRANSPORTES GENESIS

2.1 Arquitectura de referencia

2.1.1 Diagrama C4 - Nivel 1: Contexto del Sistema

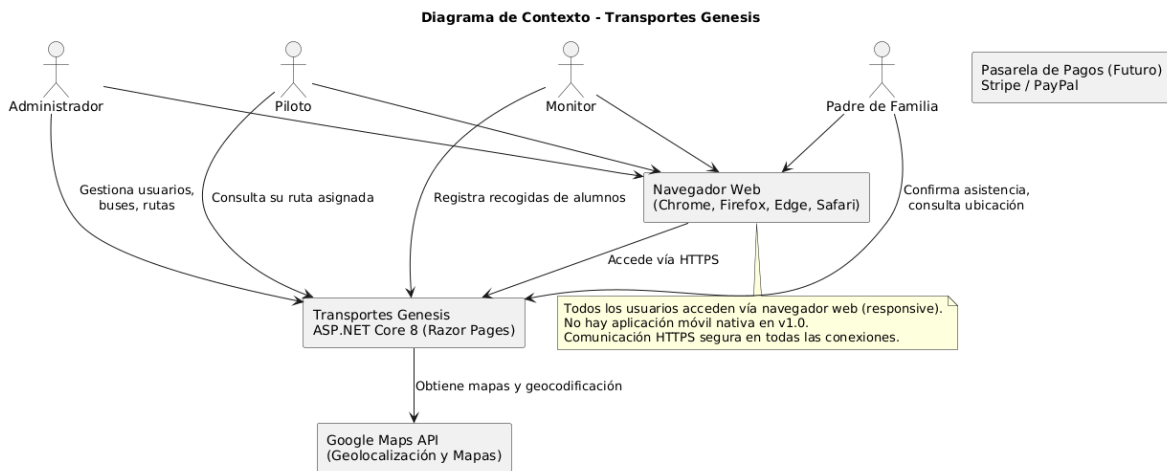
Diagrama de Contexto - Transportes Genesis

Elementos:

1. Sistema Central: "Transportes Genesis"
 - Tipo: Sistema de Software
 - Tecnología: ASP.NET Core 8 (Razor Pages)
2. Actores Externos (Personas):
 - Administrador (Gerente/Dueño)
 - Piloto (Conductor)
 - Monitor (Asistente del bus)
 - Padre de Familia (Cliente)
3. Sistemas Externos:
 - Google Maps API (Geolocalización y mapas)
 - Navegador Web (Chrome, Firefox, Edge, Safari)
 - [Futuro] Pasarela de Pagos (Stripe, PayPal)

Relaciones:

- Administrador → Transportes Genesis: "Gestiona usuarios, buses, rutas"
- Piloto → Transportes Genesis: "Consulta su ruta asignada"
- Monitor → Transportes Genesis: "Registra recogidas de alumnos"
- Padre → Transportes Genesis: "Confirma asistencia, consulta ubicación"
- Transportes Genesis → Google Maps API: "Obtiene mapas y geocodificación"
- Usuarios → Navegador Web → Transportes Genesis: "Accede vía HTTPS"



2.1.2 Diagrama C4 - Nivel 2: Contenedores

Diagrama de Contenedores - Transportes Genesis

Contenedores:

1. "Web Application" (ASP.NET Core Razor Pages)
 - Tecnología: .NET 8, Razor Pages, MVC Controllers
 - Puerto: 443 (HTTPS)
 - Responsabilidad: Interfaz de usuario, lógica de presentación
 - Componentes: Pages/, Controllers/, Views/
2. "API REST" (ASP.NET Core Web API)
 - Tecnología: .NET 8, ASP.NET Core Controllers
 - Responsabilidad: Endpoints para operaciones CRUD, cálculo de rutas
 - Ruta base: /api/
3. "SignalR Hub" (Tiempo Real)
 - Tecnología: SignalR for .NET 8
 - Responsabilidad: Comunicación bidireccional en tiempo real
 - Uso: Geolocalización de buses en vivo
4. "Base de Datos" (SQL Server)
 - Tecnología: SQL Server 2019+
 - Puerto: 1433
 - Responsabilidad: Almacenamiento persistente de datos

- Esquemas: genesis (negocio), dbo (Identity)

5. "Cliente Web" (Navegador)

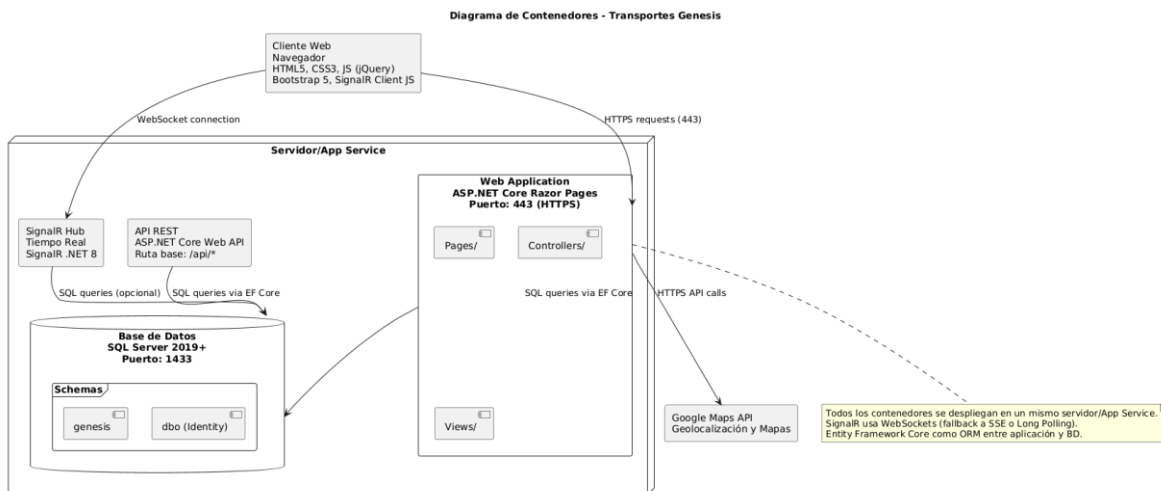
- Tecnología: HTML5, CSS3, JavaScript (jQuery)
- Frameworks: Bootstrap 5, SignalR Client JS
- Responsabilidad: Renderizado de UI, interacción con usuario

Relaciones:

- Cliente Web → Web Application: "HTTPS requests" (puerto 443)
- Cliente Web → SignalR Hub: "WebSocket connection"
- Web Application → Base de Datos: "SQL queries via EF Core"
- API REST → Base de Datos: "SQL queries via EF Core"
- SignalR Hub → Base de Datos: "SQL queries (opcional)"
- Web Application → Google Maps API: "HTTPS API calls"

Notas:

- Todos los contenedores se despliegan en un mismo servidor/App Service
- SignalR usa WebSockets (fallback a Server-Sent Events o Long Polling)
- Entity Framework Core como ORM entre aplicación y BD



2.1.3 Diagrama C4 - Nivel 3: Componentes (Web Application)

Diagrama de Componentes - Web Application

Componentes principales:

1. "Authentication & Authorization" (ASP.NET Core Identity)
 - Archivos: Controllers/AuthController.cs, Startup.cs
 - Responsabilidad: Login, logout, gestión de sesiones
 - Usa: AspNetUsers, AspNetRoles, AspNetUserRoles
2. "Pages - Admin" (Razor Pages)
 - Carpeta: Pages/Admin/
 - Páginas: Index.cshtml, CalcularRutas.cshtml, CrearUsuariosPrueba.cshtml
 - Responsabilidad: Panel de administración
3. "Pages - Piloto" (Razor Pages)
 - Carpeta: Pages/Piloto/
 - Páginas: MiRuta.cshtml
 - Responsabilidad: Vista de ruta para pilotos
4. "Pages - Monitor" (Razor Pages)
 - Carpeta: Pages/Monitor/
 - Páginas: MiRuta.cshtml, RegistrarRecogidas.cshtml
 - Responsabilidad: Vista de ruta y registro de recogidas
5. "Pages - Padres" (Razor Pages)
 - Carpeta: Pages/Padres/
 - Páginas: ConfirmarAsistencia.cshtml
 - Responsabilidad: Confirmación de asistencia de alumnos
6. "Pages - Geolocalización" (Razor Pages)
 - Carpeta: Pages/Geolocalizacion/
 - Páginas: MapaEnTiempoReal.cshtml
 - Responsabilidad: Mapa en vivo con ubicación de buses
7. "API Controllers" (MVC Controllers)
 - Archivos: Controllers/RutasController.cs, Controllers/PagosController.cs
 - Responsabilidad: Endpoints REST para operaciones
8. "SignalR Hub"
 - Archivos: Hubs/GeolocalizacionHub.cs
 - Responsabilidad: Broadcast de ubicaciones de buses
9. "Data Access Layer" (Entity Framework Core)
 - Archivos: Data/Context/ApplicationDbContext.cs
 - Responsabilidad: Acceso a base de datos, DbSets

10. "Models - Domain"

- Carpeta: Models/DB/Negocio/, Models/DB/Usuarios/
- Archivos: Bus.cs, Ruta.cs, Parada.cs, Alumnos.cs, etc.
- Responsabilidad: Entidades de dominio

11. "DTOs"

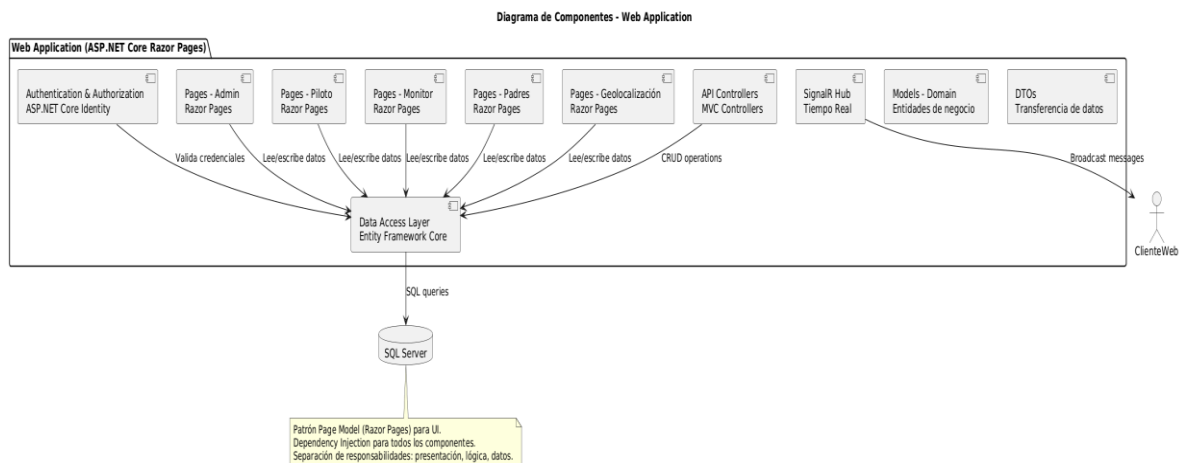
- Carpeta: DTOs/
- Archivos: RutaDto.cs, PagoDto.cs, etc.
- Responsabilidad: Transferencia de datos entre capas

Relaciones:

- Authentication → Data Access Layer: "Valida credenciales"
- Pages → Data Access Layer: "Lee/escribe datos"
- API Controllers → Data Access Layer: "CRUD operations"
- SignalR Hub → Clientes: "Broadcast messages"
- Data Access Layer → SQL Server: "SQL queries"

Notas:

- Patrón Page Model (Razor Pages) para UI
- Dependency Injection para todos los componentes
- Separation of Concerns (presentación, lógica, datos)



2.1.4 Estilo Arquitectónico y Justificación

Estilo Arquitectónico Principal

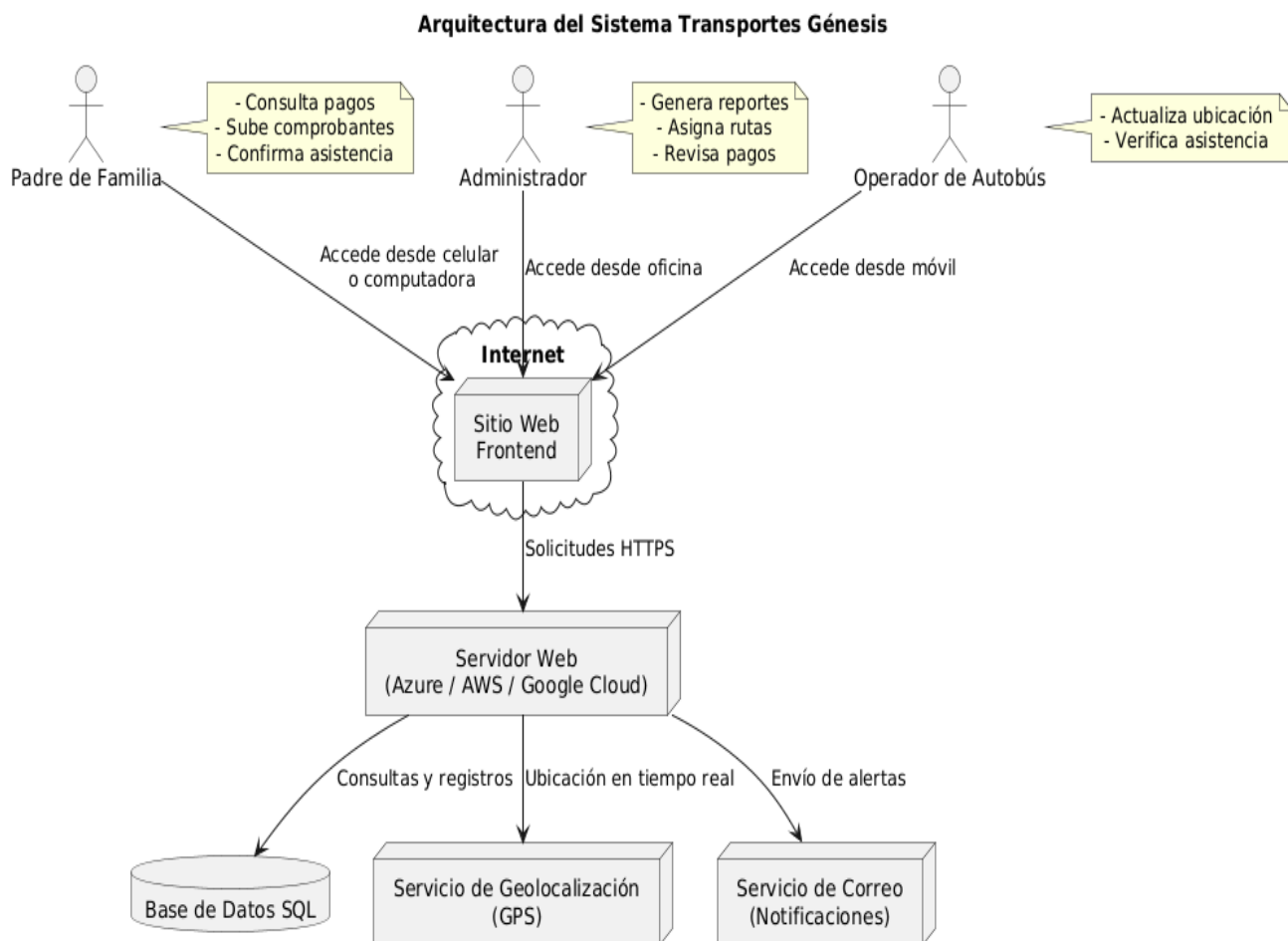
Razor Pages con Patrón Page-Focused MVC

El proyecto está construido principalmente con Razor Pages, un patrón de diseño de ASP.NET Core que simplifica el desarrollo de aplicaciones web orientadas a páginas. Este estilo se complementa con:

- MVC Controllers para endpoints API REST
- SignalR Hub para comunicación en tiempo real
- Repository Pattern implícito mediante Entity Framework Core

Capas de la Aplicación

La arquitectura sigue el siguiente flujo:



una separación de responsabilidades en 3 capas lógicas:

CAPA DE PRESENTACIÓN:

- Razor Pages (.cshtml + .cshtml.cs)
- MVC Controllers (para API)
- ViewModels / DTOs
- SignalR Hubs

CAPA DE LÓGICA DE NEGOCIO

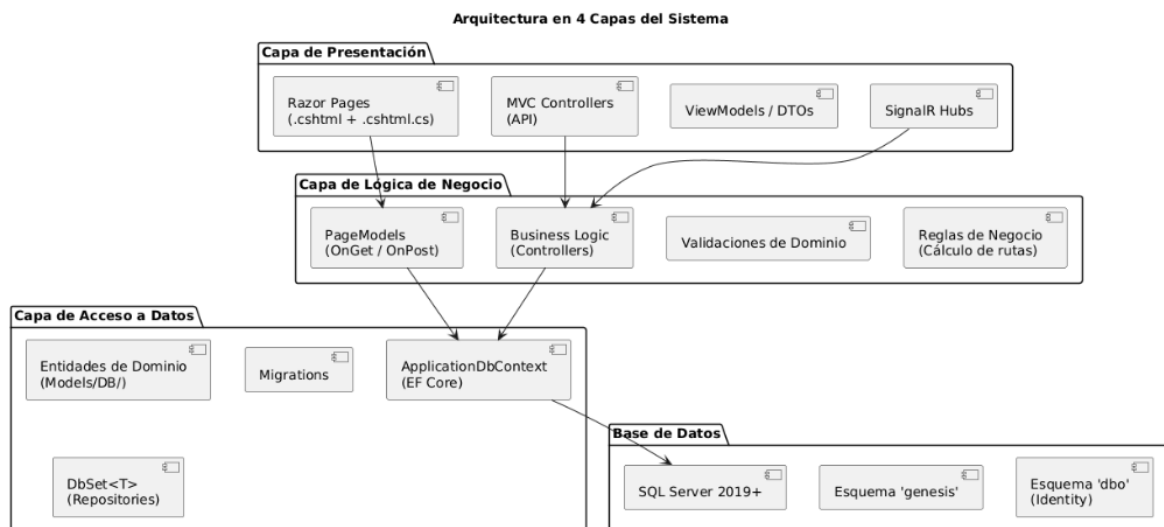
- PageModels (OnGet, OnPost methods)
- Business Logic en
- Validaciones de
- Reglas de negocio (ej: cálculo de rutas)

CAPA DE ACCESO A

- ApplicationDbContext (EF Core)
- Entidades de dominio (Models/DB/)
- Migrations
- DbSet<T> como Repositories

BASE DE

- SQL Server 2019+
- Esquema 'genesis' (negocio)
- Esquema 'dbo' (ASP.NET Core Identity)



Justificación de la Arquitectura Elegida

¿Por qué Razor Pages?

1. **Productividad:** Desarrollo rápido de páginas web sin complejidad excesiva
2. **Simplicidad:** Patrón Page-Focused más intuitivo que MVC tradicional para aplicaciones CRUD
3. **Integración nativa con .NET:** Soporte oficial de Microsoft, excelente documentación
4. **Separación de concerns:** El PageModel separa lógica de presentación (CSHTML) de lógica de negocio (CS)
5. **Madurez:** Tecnología estable y probada en producción

¿Por qué complementar con MVC Controllers?

- Para endpoints API REST (/api/*) que no requieren renderizado de UI
- Separación clara entre páginas web (Razor Pages) y servicios API (Controllers)

¿Por qué SignalR?

- **Comunicación en tiempo real necesaria para geolocalización de buses**
- **Protocolo WebSocket eficiente (bajo overhead)**
- **Integración nativa con ASP.NET Core**

2.1.5 Diagrama de Despliegue

Diagrama de Despliegue - Transportes Genesis

Nodos:

1. "Cliente" (Device)
 - Tipo: Dispositivo de usuario (PC, Tablet, Smartphone)
 - Software: Navegador Web (Chrome, Firefox, Edge, Safari)
 - Componentes:
 - HTML5/CSS3/JavaScript
 - Bootstrap 5 (UI)
 - SignalR Client (JS)
2. "Servidor Web / App Service" (Execution Environment)
 - Tipo: IIS 10 / Azure App Service
 - Sistema Operativo: Windows Server 2019+ / Azure
 - Runtime: .NET 8 Runtime
 - Componentes:
 - ASP.NET Core Application (Razor Pages + API)
 - SignalR Hub
 - Kestrel Web Server (detrás de IIS)

3. "Servidor de Base de Datos" (Database Server)
 - Tipo: SQL Server 2019+
 - Sistema Operativo: Windows Server / Azure SQL Database
 - Puerto: 1433 (TCP)
 - Componentes:
 - SQL Server Database Engine
 - Base de datos: TransportesGenesisDb
 - Esquemas: genesis, dbo
4. "Servicios Externos" (External System)
 - Google Maps API (HTTPS, puerto 443)
 - [Futuro] Pasarela de Pagos

Conexiones:

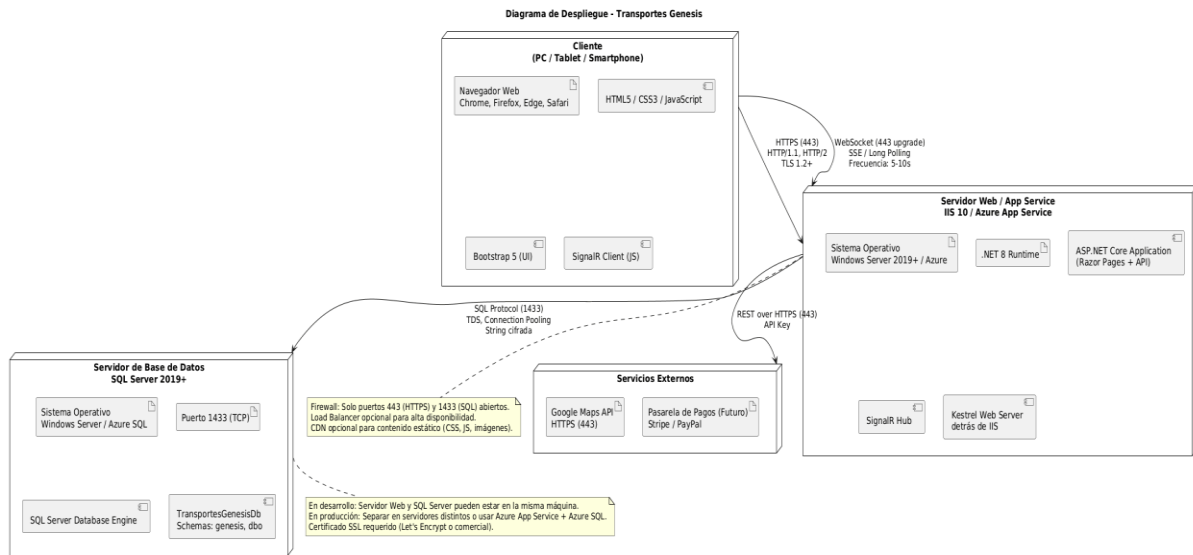
- Cliente → Servidor Web: "HTTPS (puerto 443)"
 - Protocolo: HTTP/1.1, HTTP/2
 - Seguridad: TLS 1.2+
- Cliente → SignalR Hub: "WebSocket (puerto 443 upgrade)"
 - Protocolo: WebSocket (fallback: SSE, Long Polling)
 - Frecuencia: cada 5-10 segundos
- Servidor Web → SQL Server: "SQL Protocol (puerto 1433)"
 - Protocolo: TDS (Tabular Data Stream)
 - Conexión: Connection Pooling habilitado
 - String de conexión cifrada
- Servidor Web → Google Maps API: "HTTPS (puerto 443)"
 - Protocolo: REST over HTTPS
 - Autenticación: API Key

Configuración de Red:

- Firewall: Solo puertos 443 (HTTPS) y 1433 (SQL) abiertos
- Load Balancer (opcional): Para alta disponibilidad
- CDN (opcional): Para contenido estático (CSS, JS, imágenes)

Notas:

- En desarrollo: Servidor Web y SQL Server pueden estar en la misma máquina
- En producción: Separar en servidores distintos o usar Azure App Service + Azure SQL
- Certificado SSL requerido (Let's Encrypt gratuito o comercial)



2.2 Stack tecnológico seleccionado

2.2.1 Tecnologías por Capa

Backend (Servidor)

Tecnología	Versión	Propósito	Justificación
.NET	8.0 LTS	Plataforma de desarrollo	Última versión LTS, soporte hasta 2026, alto rendimiento, multiplataforma
ASP.NET Core	8.0	Framework web	Framework moderno, cross-platform, alto rendimiento, excelente para APIs REST y Razor Pages
Razor Pages	8.0	Framework UI	Patrón simplificado para páginas web, productividad alta, menos verboso que MVC tradicional
Entity Framework Core	8.0	ORM	Acceso a datos con LINQ, migraciones automáticas, soporte para SQL Server, reduce código boilerplate

ASP.NET Core Identity	8.0	Autenticación/Autorización	Sistema completo de gestión de usuarios, roles, claims, integración nativa con EF Core
SignalR	8.0	Comunicación en tiempo real	WebSockets para geolocalización en vivo, fallback automático, integración nativa con ASP.NET Core
C#	12.0	Lenguaje de programación	Lenguaje tipado, moderno, orientado a objetos, async/await nativo, LINQ

Frontend (Cliente)

Tecnología	Versión	Propósito	Justificación
HTML5	-	Estructura de páginas	Estándar web moderno, soporte para semántica, accesibilidad
CSS3	-	Estilos y diseño	Flexbox, Grid, animaciones, variables CSS
Bootstrap	5.3	Framework CSS	UI responsive, componentes pre-diseñados, grid system, iconos, compatible con todos los navegadores
JavaScript	ES6+	Interactividad cliente	Lenguaje estándar de navegadores, soporte async/await, manipulación DOM
jQuery	3.7.0	Librería JS	Simplifica manipulación DOM, AJAX, eventos, amplia compatibilidad
SignalR Client (JS)	8.0	Cliente WebSocket	Cliente JavaScript para conexión con SignalR Hub, manejo de reconexiones

Bootstrap Icons	1.11	Iconografía	Iconos vectoriales, ligeros, consistentes con Bootstrap
------------------------	------	-------------	---

Base de Datos

Tecnología	Versión	Propósito	Justificación
SQL Server	2019+	RDBMS	Motor de base de datos empresarial, robustez, soporte de Microsoft, herramientas de administración, backups automáticos
T-SQL	-	Lenguaje de consultas	Extensión de SQL estándar, stored procedures, triggers, funciones

Infraestructura y Despliegue

Tecnología	Versión	Propósito	Justificación
IIS	10.0+	Servidor web	Servidor web de Microsoft, integración con Windows Server, soporte para .NET nativo
Kestrel	8.0	Servidor HTTP	Servidor web multiplataforma de .NET, alto rendimiento, usado internamente por ASP.NET Core
Azure App Service	-	Hosting cloud (opcional)	PaaS de Microsoft, escalado automático, integración con Azure SQL, CI/CD integrado
Windows Server	2019+	Sistema Operativo	SO estable, compatible con IIS y SQL Server, soporte empresarial

Herramientas de Desarrollo

Herramienta	Versión	Propósito
Visual Studio	2022+	IDE principal
Visual Studio Code	Latest	Editor de código alternativo
SQL Server Management Studio (SSMS)	19+	Gestión de BD
Git	2.40+	Control de versiones
GitHub	-	Repositorio remoto, colaboración
Postman	Latest	Testing de APIs
Browser DevTools	-	Debugging frontend

2.2.4 Trade-offs y Decisiones Técnicas

Decisión 1: Server-Side Rendering (Razor Pages) vs Client-Side SPA

Elegido: Server-Side Rendering (Razor Pages)

Trade-offs:

Aspecto	Server-Side (Razor)	Client-Side (SPA)
Ventajas	SEO inmediato, Desarrollo rápido, Menor complejidad, Funciona sin JS	UX más fluida, Menos carga del servidor, Interactividad rica
Desventajas	Recarga de página, Menos interactividad	SEO complejo, Mayor tamaño inicial, Requiere JS habilitado

Razón: Para v1.0, la simplicidad y rapidez de desarrollo son más importantes que una UX ultra-fluida. Se complementa con SignalR para tiempo real donde es crítico (geolocalización).

Decisión 2: Monolito vs Microservicios

Elegido: Arquitectura Monolítica (Single Deployment)

Trade-offs:

Aspecto	Monolito	Microservicios
Ventajas	Simplicidad, Deployment único, Debugging fácil, Menos overhead	Escalado independiente, Tecnologías heterogéneas, Fallos aislados
Desventajas	Escalado vertical, Coupling mayor	Complejidad operativa, Comunicación entre servicios, Debugging difícil

Razón: Para una aplicación de tamaño medio con un equipo pequeño, un monolito bien estructurado es más eficiente. Escalable a microservicios en versiones futuras si es necesario.

Decisión 3: EF Core vs Dapper vs ADO.NET

Elegido: Entity Framework Core

Trade-offs:

Aspecto	EF Core	Dapper	ADO.NET
Ventajas	Productividad alta, Migraciones automáticas, LINQ	Rendimiento excelente, Control fino	Control total, Sin overhead
Desventajas	Overhead ORM, Queries complejas lentas	No genera DDL, Más código manual	Muy verbos, Propenso a errores

Razón: EF Core ofrece el mejor balance entre productividad y rendimiento para aplicaciones CRUD. Para queries críticas se puede usar SQL raw si es necesario.

2.3 Requerimientos de infraestructura y entornos

2.3.1 Hardware Mínimo y Recomendado (On-Premise)

Servidor de Aplicación (Web Server)

Componente	Mínimo	Recomendado	Producción (Alta Disponibilidad)
CPU	2 cores, 2.0 GHz	4 cores, 2.5 GHz	8 cores, 3.0 GHz
RAM	4 GB	8 GB	16 GB
Disco	20 GB SSD	50 GB SSD	100 GB SSD (RAID 1)
Red	100 Mbps	1 Gbps	1 Gbps (redundante)
Sistema Operativo	Windows Server 2019	Windows Server 2022	Windows Server 2022 Datacenter
Software	IIS 10, .NET 8 Runtime	IIS 10, .NET 8 Runtime + SDK	IIS 10, .NET 8 Runtime, monitoreo

Servidor de Base de Datos (SQL Server)

Componente	Mínimo	Recomendado	Producción (Alta Disponibilidad)
CPU	2 cores, 2.0 GHz	4 cores, 2.5 GHz	8 cores, 3.0 GHz
RAM	8 GB	16 GB	32 GB
Disco	50 GB SSD	100 GB SSD	500 GB SSD (RAID 10)
Red	100 Mbps	1 Gbps	1 Gbps (redundante)
Sistema Operativo	Windows Server 2019	Windows Server 2022	Windows Server 2022 Datacenter
Software	SQL Server 2019 Express	SQL Server 2019 Standard	SQL Server 2019 Enterprise (Always On)

Notas: - Para desarrollo/pruebas: Servidor único con 8 GB RAM y 100 GB SSD es suficiente - Para producción: Separar servidores de aplicación y base de datos - Backups: Disco adicional de al menos 500 GB para respaldos.

2.3.3 Diagrama de Red y Topología de Seguridad

Topología de Red - Transportes Genesis (Producción)

El sistema se despliega en una arquitectura **on-premise de múltiples capas**, separando claramente los componentes de presentación, lógica de negocio y almacenamiento de datos, con el objetivo de garantizar escalabilidad, seguridad y alta disponibilidad.

En el entorno de **producción**, la infraestructura se compone de al menos dos servidores principales:

1. Servidor de Aplicación (Web Server)

Este servidor es responsable de alojar la aplicación web desarrollada en ASP.NET Core, ejecutándose sobre IIS 10 y .NET 8 Runtime. Atiende las solicitudes de los usuarios finales y gestiona la lógica de presentación y parte de la lógica de negocio.

Se recomienda que este servidor cuente con recursos suficientes (CPU multinúcleo, memoria RAM adecuada y almacenamiento SSD en RAID 1) para soportar múltiples conexiones concurrentes.

2. Servidor de Base de Datos (SQL Server)

Este servidor gestiona el almacenamiento persistente de la información del sistema mediante SQL Server. Se recomienda una configuración de alto rendimiento con mayor capacidad de memoria y almacenamiento en RAID 10 para optimizar tanto la velocidad de lectura/escritura

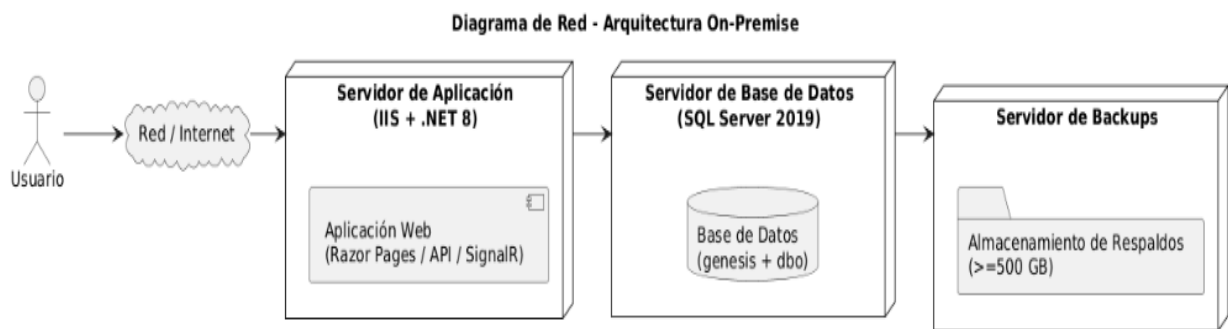
a como la tolerancia a fallos.

En entornos críticos, se sugiere el uso de **SQL Server Enterprise con Always On**, garantizando alta disponibilidad.

Ambos servidores se comunican a través de una red de alta velocidad (1 Gbps) y pueden configurarse con **redundancia de red** para evitar puntos únicos de falla.

Adicionalmente, se contempla un sistema de **respaldos (backups)** almacenado en una unidad de disco independiente, con capacidad suficiente (mínimo 500 GB), asegurando la recuperación ante desastres.

El acceso de los usuarios se realiza a través de una red local o internet, pasando por la capa de aplicación antes de llegar a la base de datos, manteniendo así la seguridad mediante aislamiento de la capa de datos.



2.4 Diseño de base de datos

2.4.2 Diagrama Entidad-Relación (ER)

Modelo Entidad-Relación - Transportes Genesis

Entidades Principales:

1. AspNetUsers (dbo) [Usuarios del sistema]
 - Id (PK, NVARCHAR(450))
 - UserName
 - Email
 - PasswordHash
 - ...campos de Identity
2. AspNetRoles (dbo) [Roles del sistema]
 - Id (PK, NVARCHAR(450))
 - Name (Administrador, Piloto, Monitor, PadreDeFamilia)

3. Bus (genesis)
 - IdBus (PK)
 - Placa (UNIQUE)
 - Modelo
 - Capacidad
 - Activo
4. Ruta (genesis)
 - IdRuta (PK)
 - IdBus (FK → Bus)
 - Nombre
 - TipoRuta ('Mañana' | 'Tarde')
 - HoraInicio
 - EsActiva
5. Parada (genesis)
 - IdParada (PK)
 - Nombre
 - Direccion
 - Latitud
 - Longitud
 - Activo
6. RutaParada (genesis) [Intermedia]
 - IdRutaParada (PK)
 - IdRuta (FK → Ruta)
 - IdParada (FK → Parada)
 - Orden
 - HoraEstimada
7. Alumnos (genesis)
 - IdAlumno (PK)
 - NombreCompleto
 - FechaNacimiento
 - Grado
 - IdPadre (FK →AspNetUsers)
 - IdParada (FK → Parada)
8. AsignacionPilotoBus (genesis)
 - IdAsignacion (PK)
 - IdUsuarioPiloto (FK →AspNetUsers)
 - IdBus (FK → Bus)
 - FechaAsignacion
 - EsActual
9. RegistroRecogida (genesis)
 - IdRegistro (PK)
 - IdParada (FK → Parada)
 - IdAlumno (FK → Alumnos)

- FechaHoraRecogida
- ConfirmadoPor (FK →AspNetUsers)
- AlumnoPresente

10. ConfirmacionAsistencia (genesis)

- IdConfirmacion (PK)
- IdAlumno (FK → Alumnos)
- Fecha
- VaAsistir
- FechaHoraConfirmacion

11. Pago (genesis)

- IdPago (PK)
- IdPadre (FK →AspNetUsers)
- Monto
- FechaPago
- MetodoPago
- Estado

Relaciones (Cardinalidades):

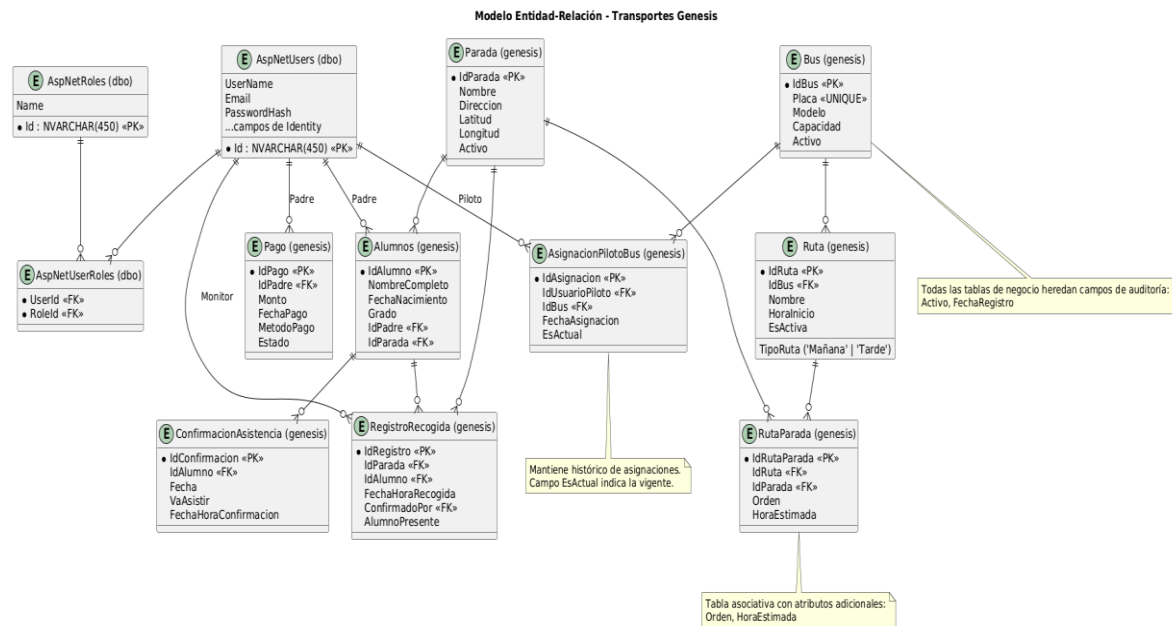
- AspNetUsers (1) ↔ (N) AspNetUserRoles ↔ (N) AspNetRoles [Many-to-Many via tabla intermedia]
- AspNetUsers (1) ↔ (N) Alumnos [Un padre puede tener muchos hijos]
- AspNetUsers (1) ↔ (N) AsignacionPilotoBus [Un piloto puede tener múltiples asignaciones históricas]
- AspNetUsers (1) ↔ (N) RegistroRecogida [Un monitor registra muchas recogidas]
- AspNetUsers (1) ↔ (N) Pago [Un padre realiza muchos pagos]
- Bus (1) ↔ (N) Ruta [Un bus tiene muchas rutas]
- Bus (1) ↔ (N) AsignacionPilotoBus [Un bus puede ser asignado a muchos pilotos]
- Ruta (1) ↔ (N) RutaParada [Una ruta tiene muchas paradas]
- Parada (1) ↔ (N) RutaParada [Una parada puede estar en muchas rutas]
- Parada (1) ↔ (N) Alumnos [Una parada puede tener muchos alumnos asignados]
- Parada (1) ↔ (N) RegistroRecogida [En una parada se registran muchas recogidas]
- Alumnos (1) ↔ (N) RegistroRecogida [Un alumno tiene muchos registros de recogidas]
- Alumnos (1) ↔ (N) ConfirmacionAsistencia [Un alumno tiene muchas confirmaciones]

Notas:

- RutaParada es una tabla asociativa con atributo adicional (Orden, HoraE

stimada)

- AsignacionPilotoBus mantiene histórico con campo "EsActual"
- Todas las tablas de negocio heredan campos de auditoría (Activo, FechaRegistro)



2.5 Diseño de APIs e integración

2.5.1 Definiciones de los APIs y Endpoints REST Principales implementados:

El sistema implementa una arquitectura basada en servicios RESTful y comunicación en tiempo real, permitiendo la interacción entre clientes (web o móviles) y el backend mediante endpoints estructurados y un hub de SignalR.

1. API REST

- La API REST se expone bajo la ruta base /api/ y permite realizar operaciones sobre los principales módulos del sistema, como rutas, pagos y confirmación de asistencia. Cada endpoint sigue el estándar HTTP, utilizando métodos como:
 - GET: para consultas de información
 - POST: para creación de recursos
- Las solicitudes requieren autenticación mediante ASP.NET Core Identity (cookies) y control de acceso basado en roles. Flujo general de una petición REST:
 - El cliente envía una solicitud HTTP al endpoint correspondiente.
 - El servidor valida la autenticación y permisos del usuario.
 - El controlador procesa la solicitud aplicando lógica de negocio.
 - Se accede a la base de datos para consultas o persistencia.

2. Módulos principales del API

- API de Rutas: permite consultar rutas activas y calcular rutas optimizadas utilizando un algoritmo tipo TSP (Traveling Salesman Problem).
 - Consulta de rutas activas por bus
 - Generación de rutas diferenciadas por turno (Mañana/Tarde)
 - Inclusión de paradas, horarios estimados y alumnos
- API de Pagos: Gestiona el registro y consulta de pagos realizados por los padres de familia.
 - Registro de pagos (estado inicial: Pendiente)
 - Consulta de historial de pagos
 - Integración futura con pasarelas de pago (Stripe)
- API de Confirmación de Asistencia: Permite a los padres confirmar la asistencia diaria de los alumnos.
 - Registro de confirmación por fecha
 - Validación de estado de asistencia

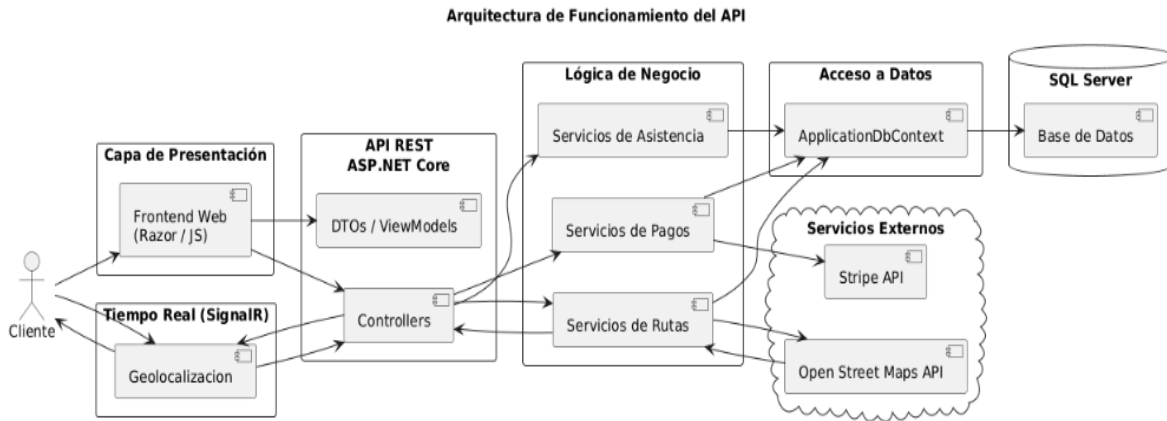
3. Comunicación en Tiempo Real (SignalR): El sistema utiliza SignalR para enviar actualizaciones en tiempo real, principalmente para el seguimiento del bus.

Flujo de funcionamiento:

- El cliente establece conexión con /geolocalizacionHub
- El cliente se suscribe al grupo "Mapa"
- El piloto envía coordenadas periódicamente
- El servidor transmite la ubicación a todos los clientes conectados
- Los clientes actualizan el mapa en tiempo real.

4. Integración con Servicios Externos

- Street Maps API: usado para:
 - Visualización de mapas interactivos
 - Geolocalización
 - Marcadores de buses
- Stripe (No implementado en v1.0): diseñado para:
 - Procesamiento de pagos en línea
 - Tokenización segura de tarjetas



2.6 Patrones de diseño y buenas prácticas

2.6.1 Estrategia de Testing

Testing Unitario (Unit Tests)

Requerimientos de Pruebas

Los requerimientos funcionales y no funcionales que deben ser validados incluyen:

- Registro y control de pagos.
- Carga de comprobantes bancarios.
- Geolocalización en tiempo real.
- Confirmación de asistencia por parte de los padres.
- Generación de reportes de rutas optimizadas.
- Restricción de pagos dentro de los primeros 5 días del mes.
- Acceso restringido a usuarios registrados.
- Estrategia de Pruebas

Objetivo:

- Validar que el sistema cumpla con los requerimientos funcionales y no funcionales definidos, garantizando estabilidad, seguridad y usabilidad para los usuarios finales.

Técnica

- Pruebas funcionales: Validación de cada módulo según los casos de uso.
- Pruebas de integración: Verificar la interacción entre módulos (ej. pagos y reportes).
- Pruebas de aceptación: Simulación de uso por parte de padres y administración.
- Pruebas de rendimiento: Evaluar tiempos de respuesta en geolocalización y carga de comprobantes.

Criterio de Terminación

- Todos los casos de prueba deben estar ejecutados.
- Al menos el 95% de los casos deben pasar exitosamente.
- Los errores críticos deben estar corregidos.
- Validación por parte del solicitante (Familia Villedas).

Recursos Necesarios

- Equipo de pruebas: 2 estudiantes (Jonathan Samuel Villeda Pérez y José David Florian).
- Ambiente de pruebas: Pruebas.
- Lenguajes involucrados: JavaScript, VB.NET.
- Herramientas de pruebas: Navegador web, simulador GPS, base de datos de prueba.

Testing End-to-End (E2E)

Herramienta: Selenium WebDriver o Playwright

Ejemplo de escenario:

1. Usuario navega a /Auth/Login
2. Ingresa credenciales de piloto
3. Click en "Iniciar Sesión"
4. Verifica redirección a /Piloto/MiRuta
5. Verifica que aparece "Bus #1" en la página.

2.7 Plan de implementación y roadmap técnico

2.7.1 Fases del Proyecto (Completadas)

Fase	Duración	Módulos Implementados	Estado	Fecha
Fase 0: Setup Inicial	0.5 semana	Configuración de proyecto, BD, Identity	COMPLETADO	Abril 2026
Fase 1: MVP Core	0.5 semanas	Login, Roles, Gestión de usuarios	COMPLETADO	Abril 2026
Fase 2: Rutas	1 semanas	CRUD de buses, paradas, cálculo de rutas (TSP)	COMPLETADO	Abril 2026
Fase 3: Geolocalización	2 semanas	SignalR Hub, Mapa en tiempo real	COMPLETADO	Mayo 2026
Fase 4: Pagos y Asistencia	1 semanas	Módulo de pagos, Confirmación de asistencia	COMPLETADO	Mayo 2026
Fase 5: Área de Monitores	1 semanas	Registro de recogidas, Área del Monitor	COMPLETADO	Mayo 2026
Fase 6: Testing y Deploy	1 semanas	Testing, Documentación, Despliegue	EN CURSO	Mayo 2026

Total: ~ 7 semanas (2 meses aproximado)

2.8 Análisis de riesgos técnicos y plan de mitigación

2.8.1 Matriz de Riesgos Técnicos

ID	Riesgo	Probabilidad	Impacto	Severidad	Mitigación	Contingencia
RT-01	Pérdida de conexión SignalR (WebSocket)	Media (40%)	Alto	 Alta	- Implementar reconexión automática- Fallback a Server-Sent Events- Fallback a Long Polling- Timeout de 30 segundos	- Mostrar mensaje "Reconectando..."- Permitir uso sin tiempo real (solo refresh manual)
RT-02	Bug en algoritmo TSP con muchas paradas (>30)	Media (30%)	Medio	 Media	- Tests unitarios exhaustivos- Límite de 30 paradas por ruta- Timeout de 60 segundos en cálculo- Algoritmo heurístico (nearest neighbor)	- Calcular ruta de forma manual- Dividir ruta en sub-rutas- Usar servicios externos (Google Directions API)
RT-03	Vulnerabilidades de seguridad (XSS, CSRF, SQL Injection)	Media (30%)	Crítico	 Alta	- Validación de entrada (Data Annotations)- Anti-CSRF tokens automáticos (Razor)- Parametrización de queries (EF Core)- Content Security Policy	- Aplicar parches inmediatamente- Notificar a usuarios- Cambiar contraseñas forzosamente- Contratar auditoría externa

					headers- Auditorías de seguridad trimestrales	
RT-04	Pérdida de datos por fallo de disco	Muy Baja (5%)	Crítico	● Media	- Backups automáticos diarios- Retención de 30 días- Geo- redundancia (Azure)- RAID 10 en on-premise	- Restaurar desde backup más reciente- Máximo pérdida: 24 horas de datos- Proceso de restauración documentado
RT-05	Rendimiento degradado en horas pico	Media (35%)	Medio	● Media	- Load balancer con múltiples instancias- Auto-scaling habilitado- CDN para contenido estático- Output caching en páginas estáticas	- Escalar manualmente más instancias- Priorizar funciones críticas (login, mapa)- Mensaje: “Alta demanda, procesando...”
RT-06	Errores de migración de base de datos	Baja (15%)	Alto	● Media	- Revisar migraciones antes de aplicar- Backup antes de cada migración- Probar en staging primero- Rollback script preparado	- Rollback a versión anterior- Aplicar migración manualmente- Hotfix deployment

Leyenda:

- **Severidad Alta:** Requiere acción inmediata
- **Severidad Media:** Monitorear de cerca
- **Severidad Baja:** Aceptable con mitigación básica

3. ESPECIFICACIÓN ECONÓMICA

3.1 Resumen Ejecutivo Económico

El proyecto de desarrollo del sistema de gestión de transporte escolar para la empresa Transportes Génesis presenta una viabilidad económica favorable, al estar orientado principalmente a la optimización de procesos internos y mejora del servicio al cliente.

La inversión inicial estimada asciende a **Q32,970.00**, cubriendo desarrollo, infraestructura básica, capacitación y mantenimiento inicial. El proyecto no está enfocado en generar ingresos inmediatos, sino en reducir costos operativos, minimizar errores y mejorar la eficiencia administrativa.

Se proyecta que la inversión podrá recuperarse en un periodo aproximado de **10 meses**, debido a la reducción de tiempo administrativo, mejora en la organización y aumento en la satisfacción del cliente.

3.2 Análisis de Costos (Cost Breakdown Structure)

3.2.1 Costos de Desarrollo

Recurso	Tiempo estimado	Costo por hora	Total
2 desarrolladores	160 horas	Q75	Q24,000
Diseñador UI/UX	40 horas	Q60	Q2,400
Tester QA	40 horas	Q60	Q2,400
Subtotal			Q28,800

3.2.2 Costos de Infraestructura

Servicio	Costo mensual	Tiempo	Total
Hosting Web	Q150	6 meses	Q900
Base de Datos SQL	Q100	6 meses	Q600
Dominio Web	—	1 año	Q120
Subtotal			Q1,620

3.2.3 Costos de Capacitación

Recurso	Tiempo	Costo por hora	Total
Capacitación	10 hrs	Q75	Q750
Manual usuario	—	—	Q400
Subtotal			Q1,050

3.2.4 Costos de Mantenimiento

Recurso	Tiempo	Costo por hora	Total
Técnico	20 hrs	Q75	Q1,500
Subtotal			Q1,500

3.2.5 Total General del Proyecto

Concepto	Total
Desarrollo	Q28,800.00
Infraestructura	Q1,620.00
Capacitación	Q1,050.00
Mantenimiento	Q1,500.00
Total General	Q32,970.00

3.3 Estimación de Esfuerzo y Duración

Se utiliza un enfoque basado en **estimación por horas (similar a Story Points convertidos a tiempo)**.

- Duración total estimada: **8 a 10 semanas**
- Esfuerzo total: **240 horas**

Rango de estimación:

Escenario	Tiempo estimado
P50	240 horas
P80	260 horas
P90	280 horas

Se considera una variación máxima del **±10%** según complejidad técnica.

3.4 Modelo de Negocio

Tipo de modelo:

- Sistema interno (uso empresarial)
- No orientado inicialmente a venta directa

Propuesta de valor:

- Automatización de procesos
- Mejora en control de rutas y pagos
- Seguimiento en tiempo real del transporte

Estrategia futura:

- Escalabilidad hacia otros clientes
- Posible modelo SaaS a largo plazo

3.5 Análisis Financiero

Flujo de retorno (estimado)

Debido a que no se generan ingresos directos, el retorno se mide en ahorro operativo:

- Reducción de tiempo administrativo: 30–40%
- Reducción de errores: 50%
- Mejora en gestión operativa

Indicadores:

- **Payback estimado:** 10 meses
- **ROI:** Alto en términos operativos (no monetarios inmediatos)
- **Break-even:** Alcanzado en el primer año

3.6 Análisis de Mercado y Competencia

Contexto:

El sistema está dirigido a una empresa familiar ya operativa, por lo que no compite directamente en un mercado abierto.

Oportunidad:

- Digitalización de servicios tradicionales
- Diferenciación frente a otros transportistas escolares

3.7 Plan de Negocio Ejecutivo

El proyecto se enfoca en:

- Modernización del servicio
- Mejora de la experiencia del cliente
- Optimización de procesos internos
- Base para crecimiento futuro

Se prioriza la eficiencia operativa sobre la generación inmediata de ingresos.

3.8 Fuentes de Financiamiento

- Inversión privada (empresa familiar)
- Posible reinversión de ingresos actuales

No se contempla financiamiento externo en la fase inicial.

3.9 Análisis de Riesgos Económicos

Riesgo	Probabilidad	Impacto	Mitigación
Sobrecosto de desarrollo	Media	Medio	Control de alcance
Incremento en hosting	Baja	Bajo	Uso de planes escalables
Baja adopción del sistema	Media	Alta	Capacitación adecuada
Fallas técnicas	Baja	Medio	Mantenimiento o periódico

Beneficios del Proyecto

Beneficios Tangibles

- Automatización de pagos
- Reducción de errores
- Generación automática de reportes
- Ahorro de tiempo administrativo

Beneficios Intangibles

- Mejora de la imagen empresarial
- Mayor confianza de los padres
- Mejor comunicación con usuarios